

머신러닝 및 실습

(Lecture 10)

노연수

(esnoh@koreatech.ac.kr)

Course Plan

Week	Lecture	Contents	Notes
1	-	-	대체휴일
2	파이썬 기초 I	자료형, 연산자, 조건문, 반복문	
3	파이썬 기초 II	함수, 모듈, 패키지, 클래스	
4	머신러닝이란 무엇인가?	머신러닝 기초 개념, 머신러닝 시스템의 종류 등	
5	머신러닝의 전체적인 흐름 이해하기 I	프로젝트 방향 수립, 데이터 수집, 데이터 분석	
6	머신러닝의 전체적인 흐름 이해하기 II	데이터 전처리, 모델 학습, 모델 튜닝	
7	분류모델의 이해	이진 분류기 훈련, 성능 측정, 다중 분류	
8	중간고사	-	
9	회귀모델의 이해	선형 회귀, 경사 하강법	프로젝트 공지
10	회귀모델의 확장	다항 회귀, 로지스틱 회귀	
11	서포트 벡터 머신의 이해	규제 선형 모델, SVM 분류/회귀 등	
12	다양한 분류 모델과 앙상블	결정 트리, 랜덤 포레스트	
13	차원 축소	차원 축소, 주성분 분석(PCA)	이러닝(5/25)
14	군집 분석	K-Means clustering	
15	프로젝트 발표	프로젝트 발표	
16	보강, 기말고사	Week 1 보강(6/15 0900-1100), 기말고사(6/17, 1600-1800)	

결정 트리(Decision Tree)

분류 문제에서의 결정 트리

고양이 분류 이진 문제

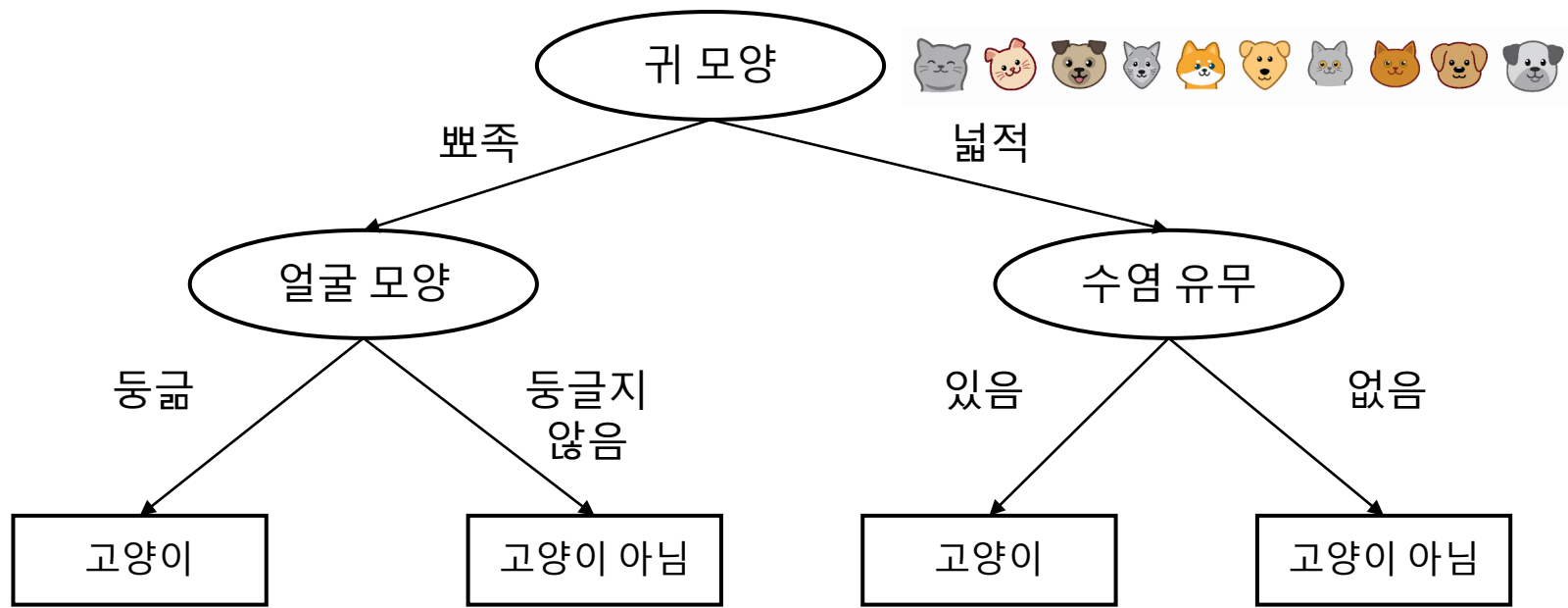
	귀 모양	얼굴 모양	수염 유무	고양이
	뾰족	둥글	있음	1
	넓적	둥글	있음	1
	넓적	둥글	없음	0
	뾰족	둥글지 않음	있음	0
	뾰족	둥글	없음	1
	넓적	둥글지 않음	없음	0
	뾰족	둥글	있음	1
	뾰족	둥글	없음	1
	넓적	둥글	없음	0
	넓적	둥글	없음	0

결정 트리(Decision Tree)

개념

▶ 특징에 따라 데이터를 분할(split)하여 최종 예측을 도출하는 지도 학습 모델

- 루트 노드(root node): 트리에서 처음 시작하는 노드
- 가지(branch): 조건에 따라 이동하는 경로
- 결정 노드(decision node): 특징을 기준으로 가지를 나누는 노드 (분할 노드)
- 리프 노드(leaf node): 최종 예측값을 출력하는 노드



결정 트리(Decision Tree)

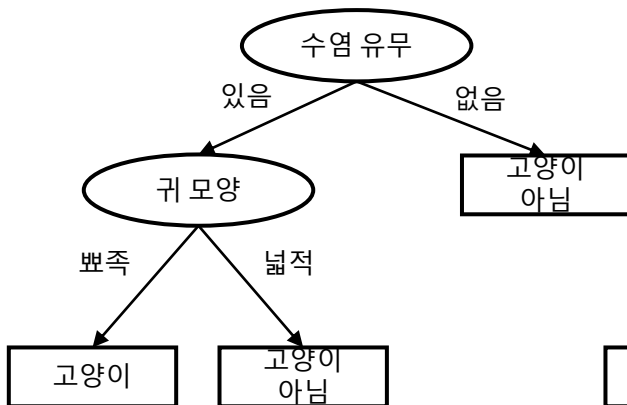
결정 트리의 형태

- ▶ 같은 데이터에서 여러 가지 형태의 결정 트리가 만들어질 수 있음

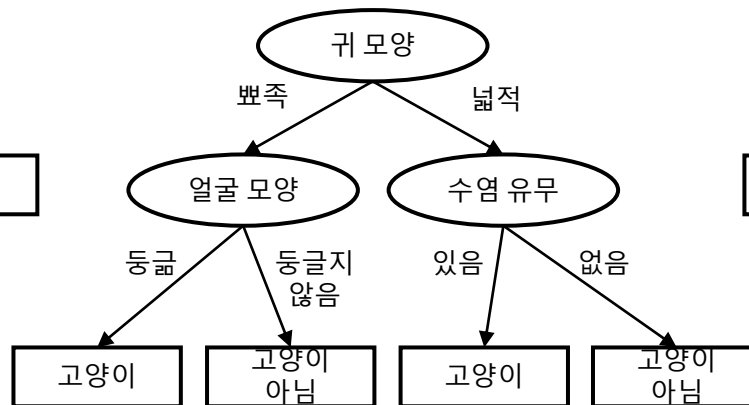
훈련 목표: 다양한 트리 중에서 가장 좋은 트리를 찾는 것



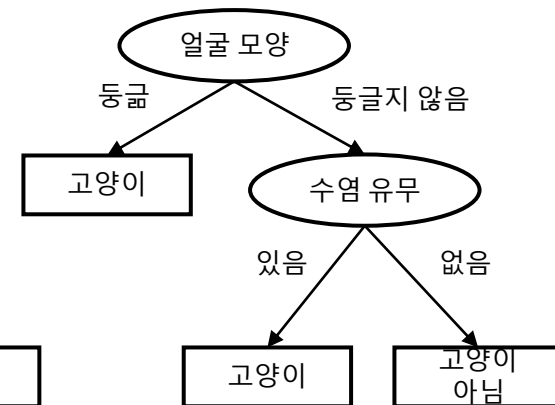
결정 트리 1



결정 트리 2



결정 트리 3

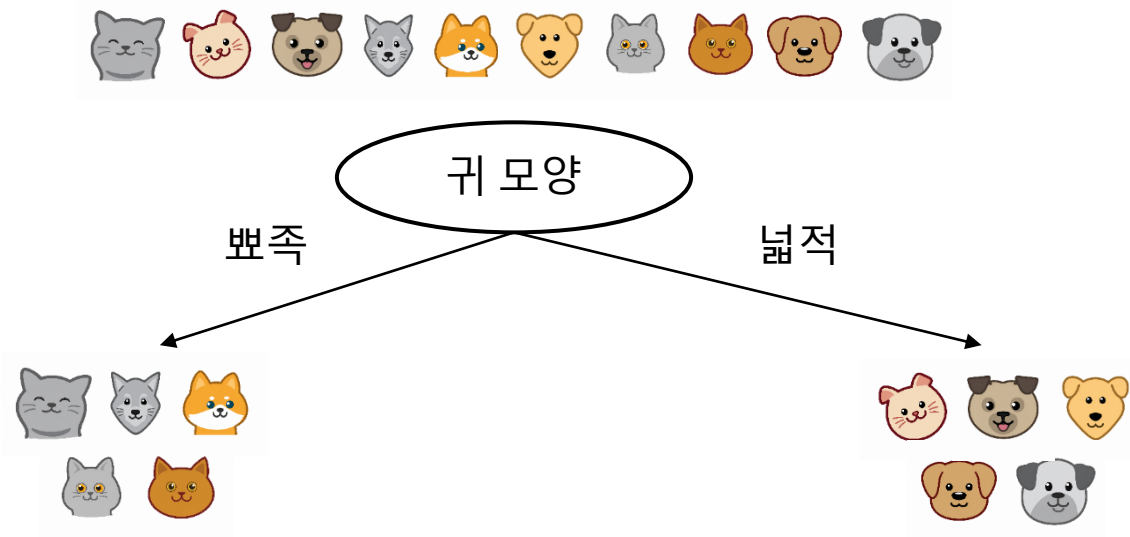


결정 트리(Decision Tree)

I 결정 트리 훈련(1)

▶ 루트 노드에서 어떤 특징을 사용할지 결정

- 전체 데이터를 특징 1개 기준으로 분할(split)

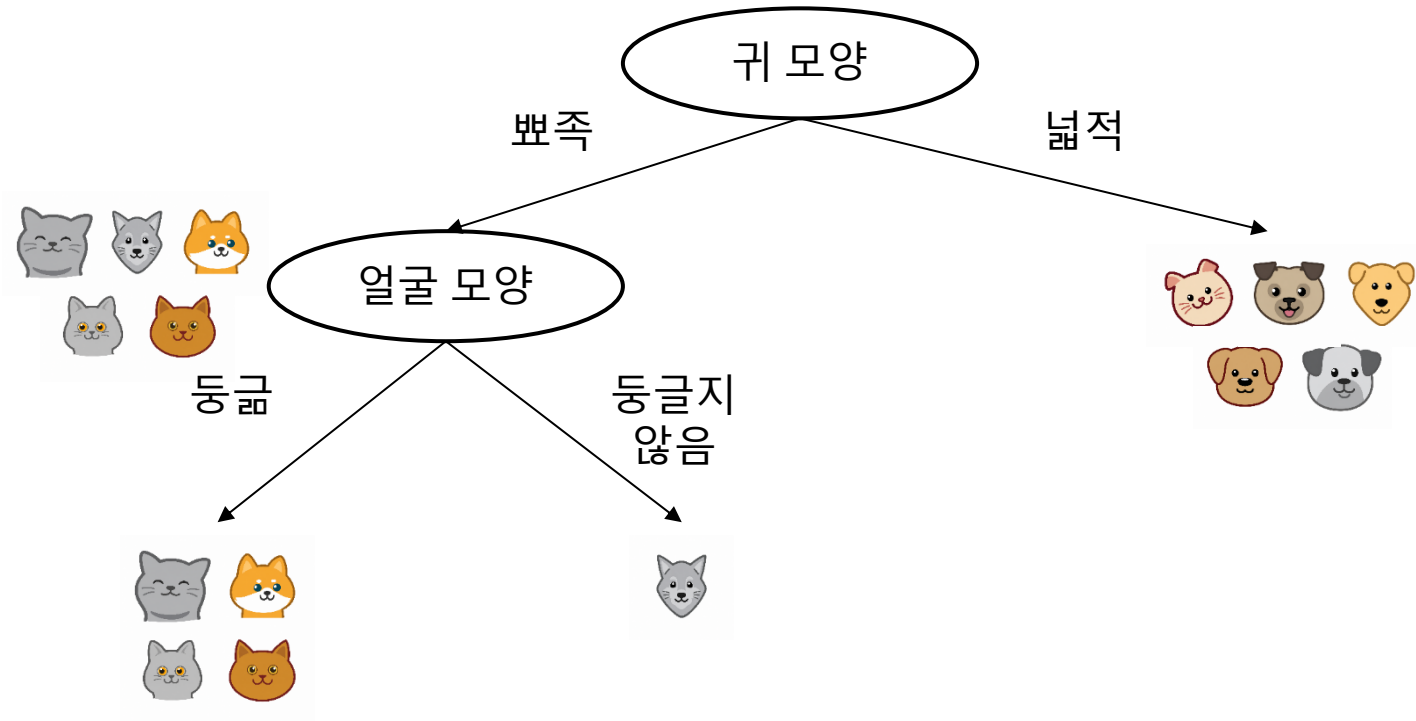


결정 트리(Decision Tree)

결정 트리 훈련(2)

▶ 각 가지(branch)에서 데이터를 분할(split)하기 위한 특징 기준 선택

- 예시: 왼쪽 가지에서 얼굴 모양을 기준으로 다시 나눔

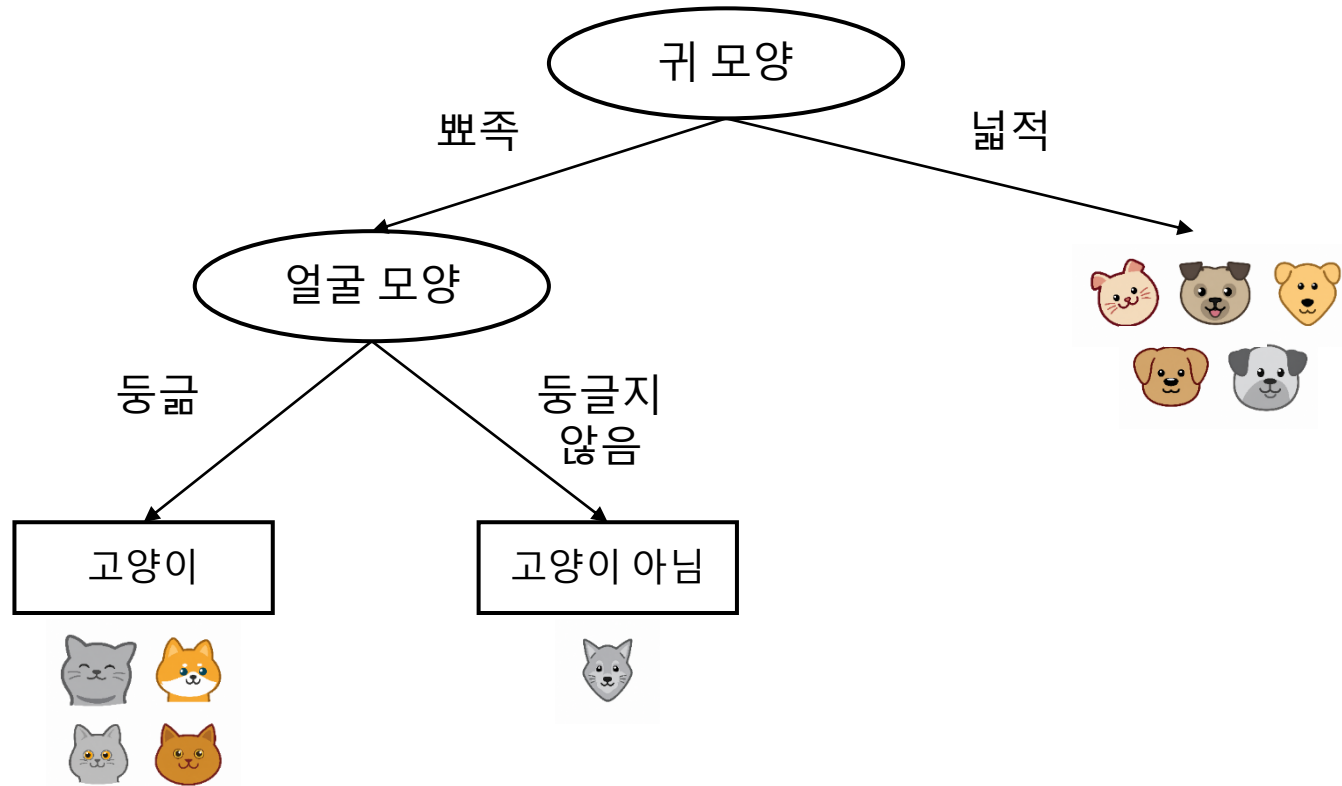


결정 트리(Decision Tree)

I 결정 트리 훈련(3)

▶ 노드에 같은 클래스만 남으면 리프 노드(leaf node)를 만듦

- 리프 노드에 도달한 샘플은 해당 클래스로 예측

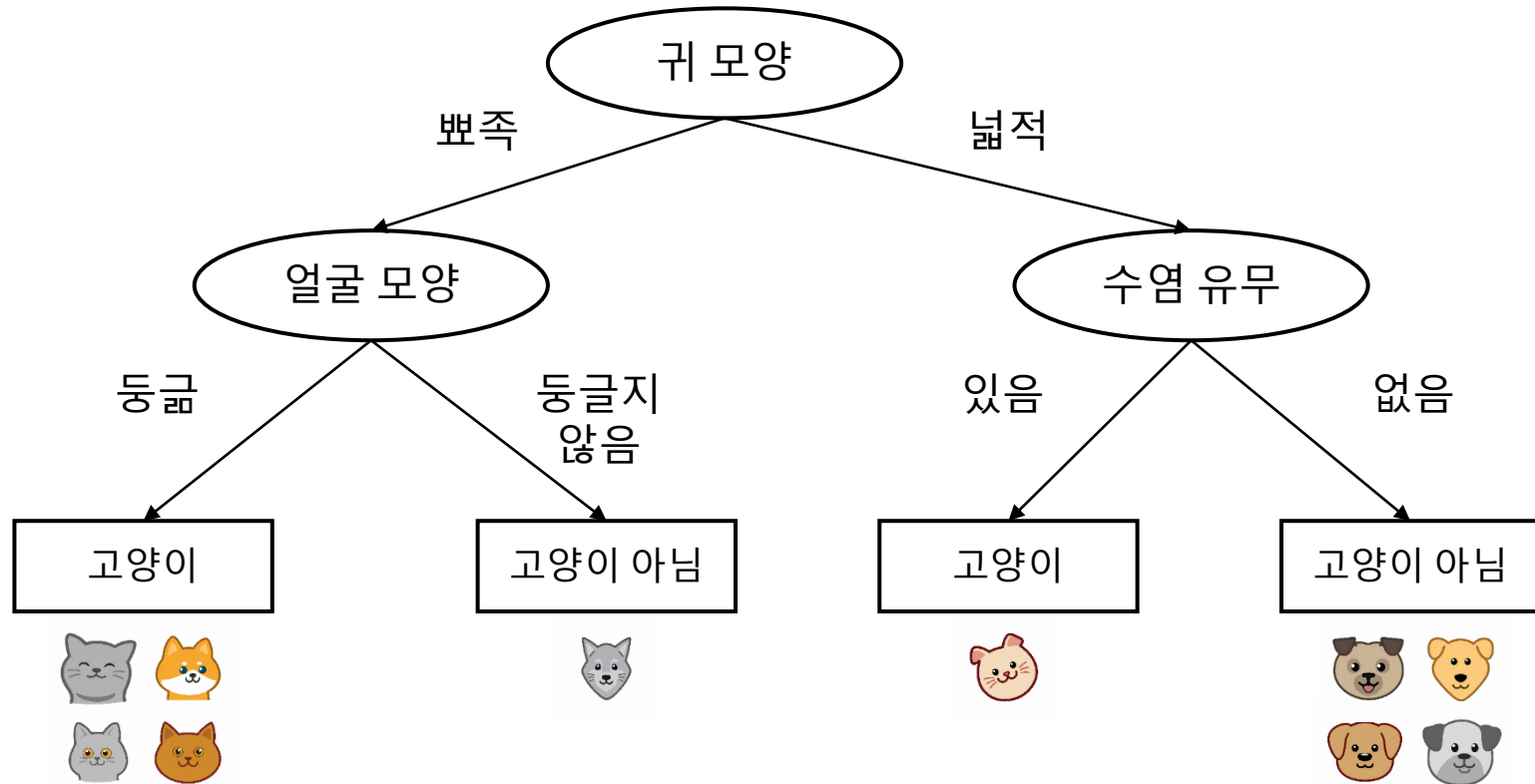


결정 트리(Decision Tree)

결정 트리 훈련(4)

▶ 오른쪽 가지도 같은 방식으로 반복

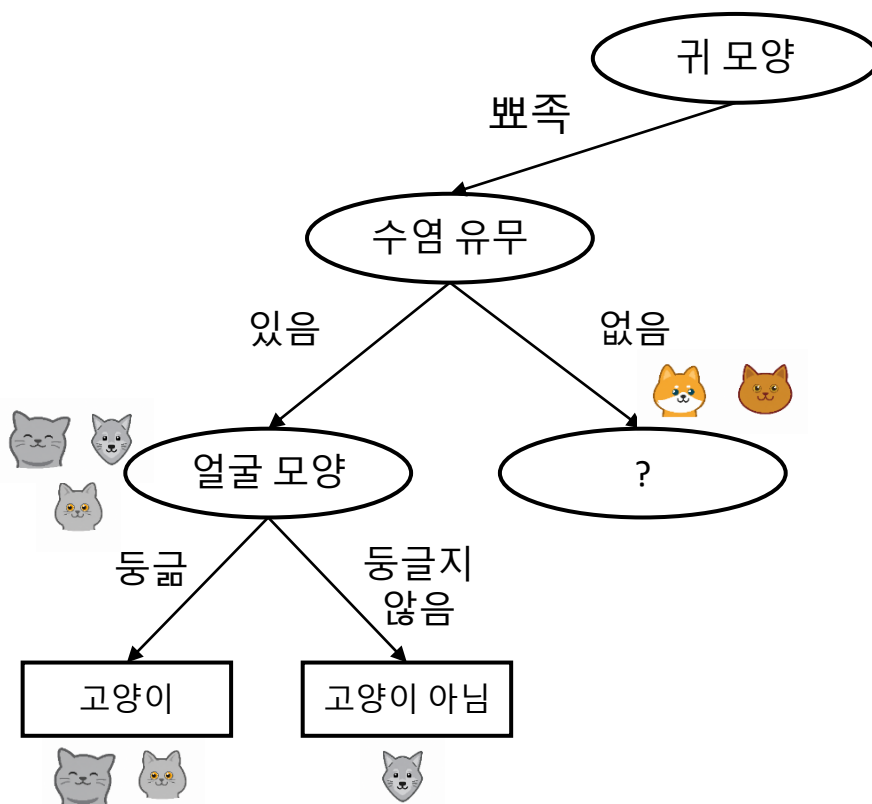
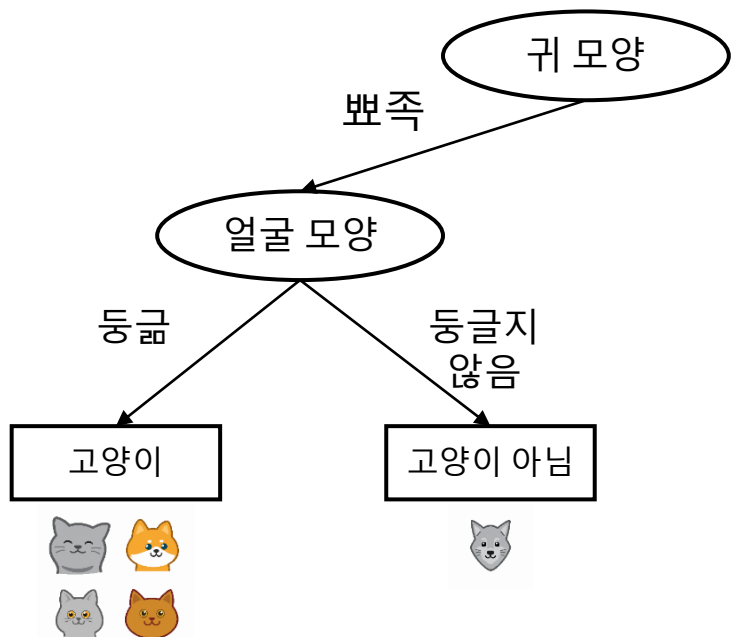
- 예시: 수염 유무에 따라 분할(splitting)



결정 트리(Decision Tree)

어떤 특징으로 분할할 것인가? (1)

▶ 훈련 알고리즘은 각 노드에서 데이터를 가장 잘 구분하는 특징을 선택함

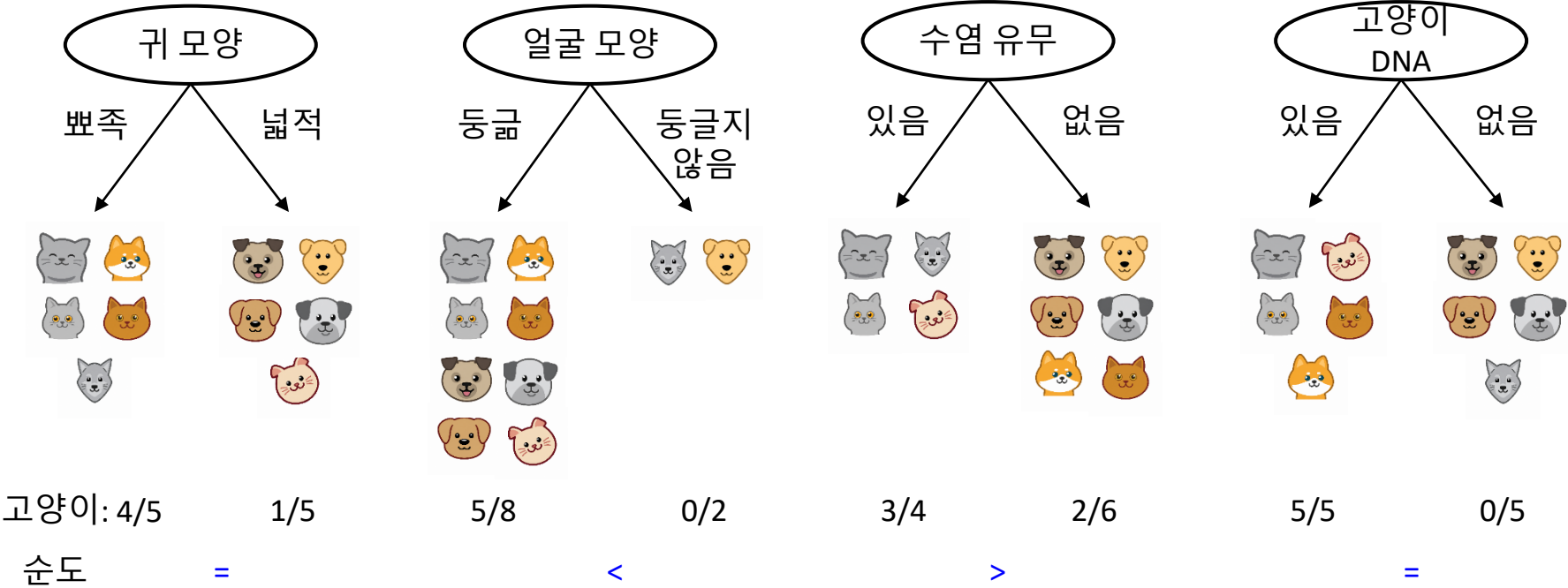


결정 트리(Decision Tree)

어떤 특징으로 분할할 것인가? (2)

▶ 좋은 분할(split)은 순도(purity)를 높이는 분할

- 순도(purity): 한 노드 안에 같은 클래스가 얼마나 잘 모여 있는지를 의미함
- 순도가 높다 → 데이터가 잘 구분되어 있다

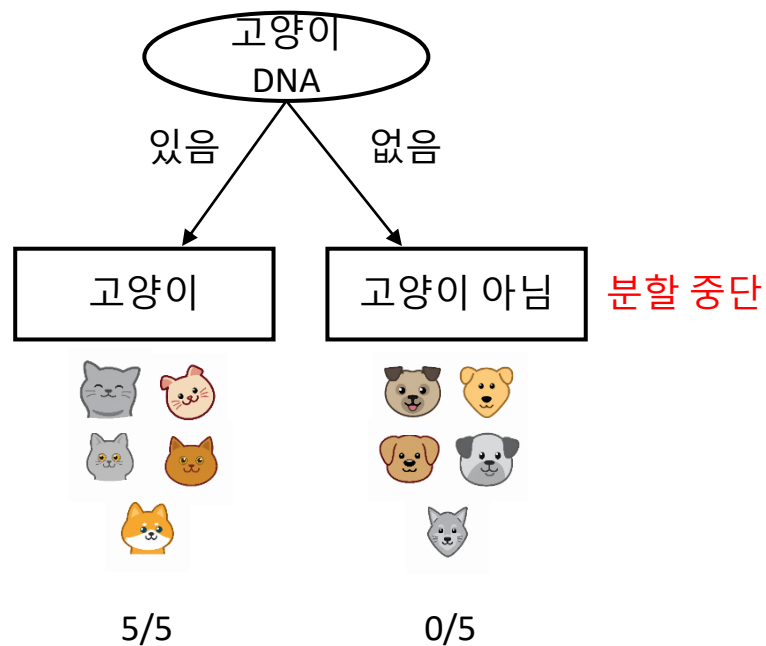


결정 트리(Decision Tree)

언제 분할(splitting)을 멈출 것인가? (1)

▶ 기준 1: 클래스가 하나만 남으면 멈춤

- 모두 고양이 → 고양이 리프 노드
- 모두 강아지 → 고양이 아님 리프 노드

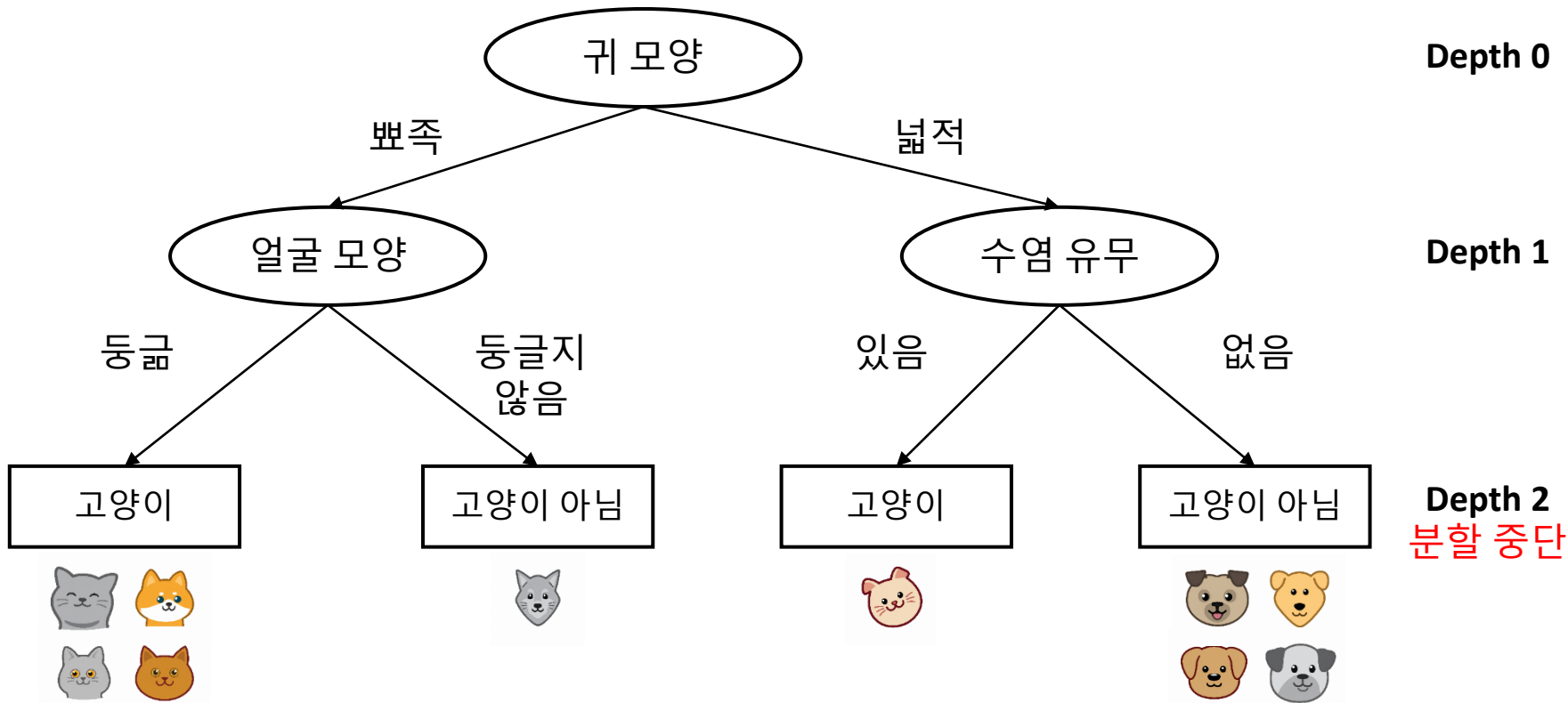


결정 트리(Decision Tree)

언제 분할(splitting)을 멈출 것인가? (2)

▶ 기준 2: 최대 깊이(depth)에 도달하면 멈춤

- 트리를 작게 유지 → 과적합 줄임

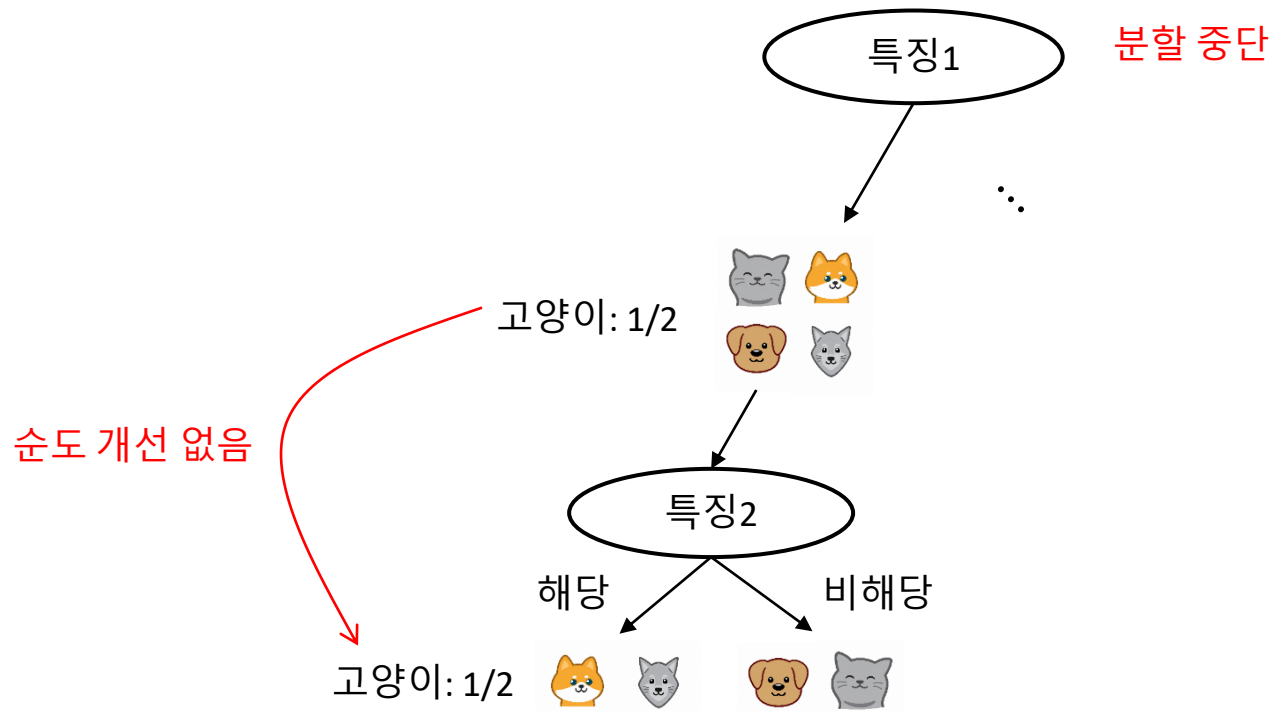


결정 트리(Decision Tree)

언제 분할(splitting)을 멈출 것인가? (2)

▶ 기준 3: 순도 개선이 작으면 멈춤

- 분할 후에도 클래스 비율이 변하지 않으면 추가로 분할하지 않음
- 불필요한 분할을 줄여 트리를 작게 유지 → 과적합 완화



언제 분할(splitting)을 멈출 것인가? (2)

▶ 기준 4: 노드의 샘플 수가 너무 적은 경우

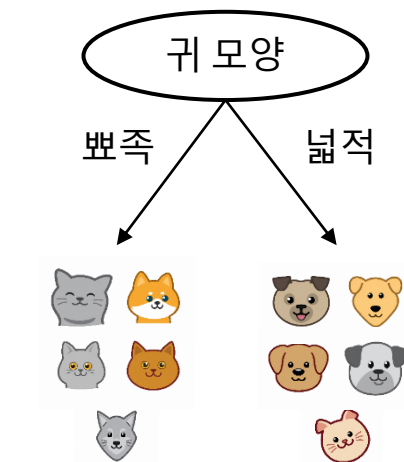
- 분할을 멈추고 다수 클래스를 기준으로 리프 노드를 만듦
- 예: 고양이(1), 강아지(2) → 고양이 아님으로 예측하는 리프 노드



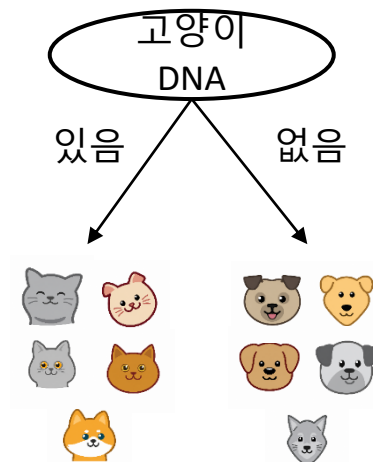
순도 측정에 대한 필요성

▶ 두 클래스가 섞여 있을때, 얼마나 순수(pure)한지 수치로 표현할 수 있을까?

- 고양이만 있음 → 순도가 높다
- 강아지만 있음 → 순도가 높다
- 고양이와 강아지가 섞여 있으면? → ?



고양이: 4/5 1/5
순도 ? = ?



5/5 0/5
? = ?

엔트로피(1)

▶ 데이터 집합의 불순도(impurity)를 측정하는 함수

- 엔트로피가 낮다 → 불순도가 낮다 → 순도가 높다
- 엔트로피가 높다 → 불순도가 높다 → 순도가 낮다

$$H = - \sum_{k=1}^n p_k \log_2(p_k)$$

$$p_k = \frac{\text{클래스 } k \text{에 속한 샘플 수}}{\text{전체 샘플 수}}$$

n = 클래스 수

이진 분류 문제  $n = 2$

$$H = -p_1 \log_2(p_1) - p_2 \log_2(p_2)$$

$$p_1 = \frac{\text{양성 샘플 수}}{\text{전체 샘플 수}}$$

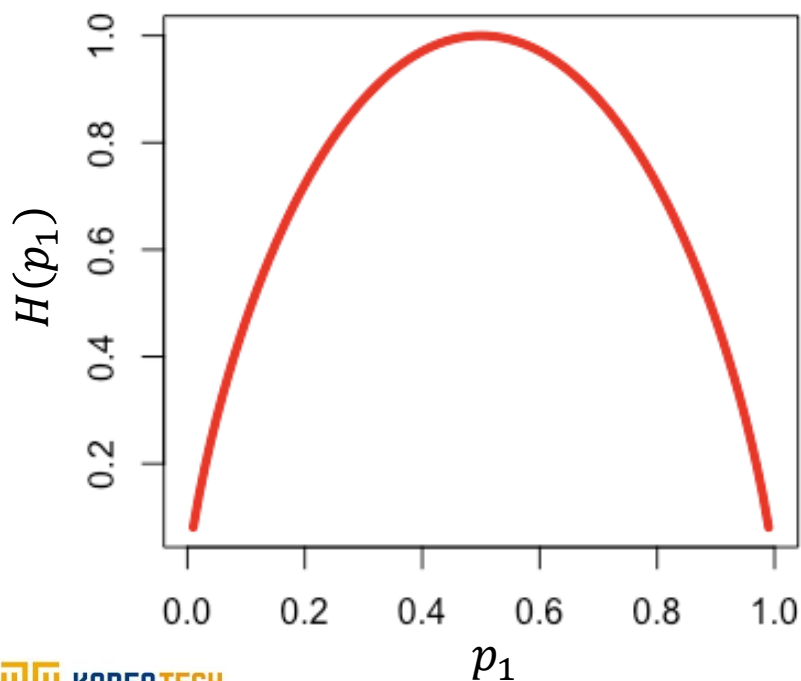
$$p_2 = \frac{\text{음성 샘플 수}}{\text{전체 샘플 수}}$$

엔트로피(2)

▶ 이진 분류 문제에서 클래스 비율이 5:5인 경우 엔트로피가 최대

- 클래스가 1개 → 엔트로피 0

$$\begin{aligned} H &= -p_1 \log_2(p_1) - p_2 \log_2(p_2) \\ &= -p_1 \log_2(p_1) - (1 - p_1) \log_2(1 - p_1) \end{aligned}$$



고양이 샘플 수	고양이 아닌 샘플 수	p_1	$H(p_1)$
2	4	2/6	0.92
3	3	3/6	1
5	1	5/6	0.65
6	0	6/6	0
0	6	0	0

지니 불순도(gini impurity)

- ▶ 한 노드 안에 여러 클래스가 얼마나 섞여 있는지 나타내는 지표 (엔트로피와 유사)
 - 클래스가 1개 → 지니 불순도 0 (순수함)

$$G = 1 - \sum_{k=1}^n p_k^2$$

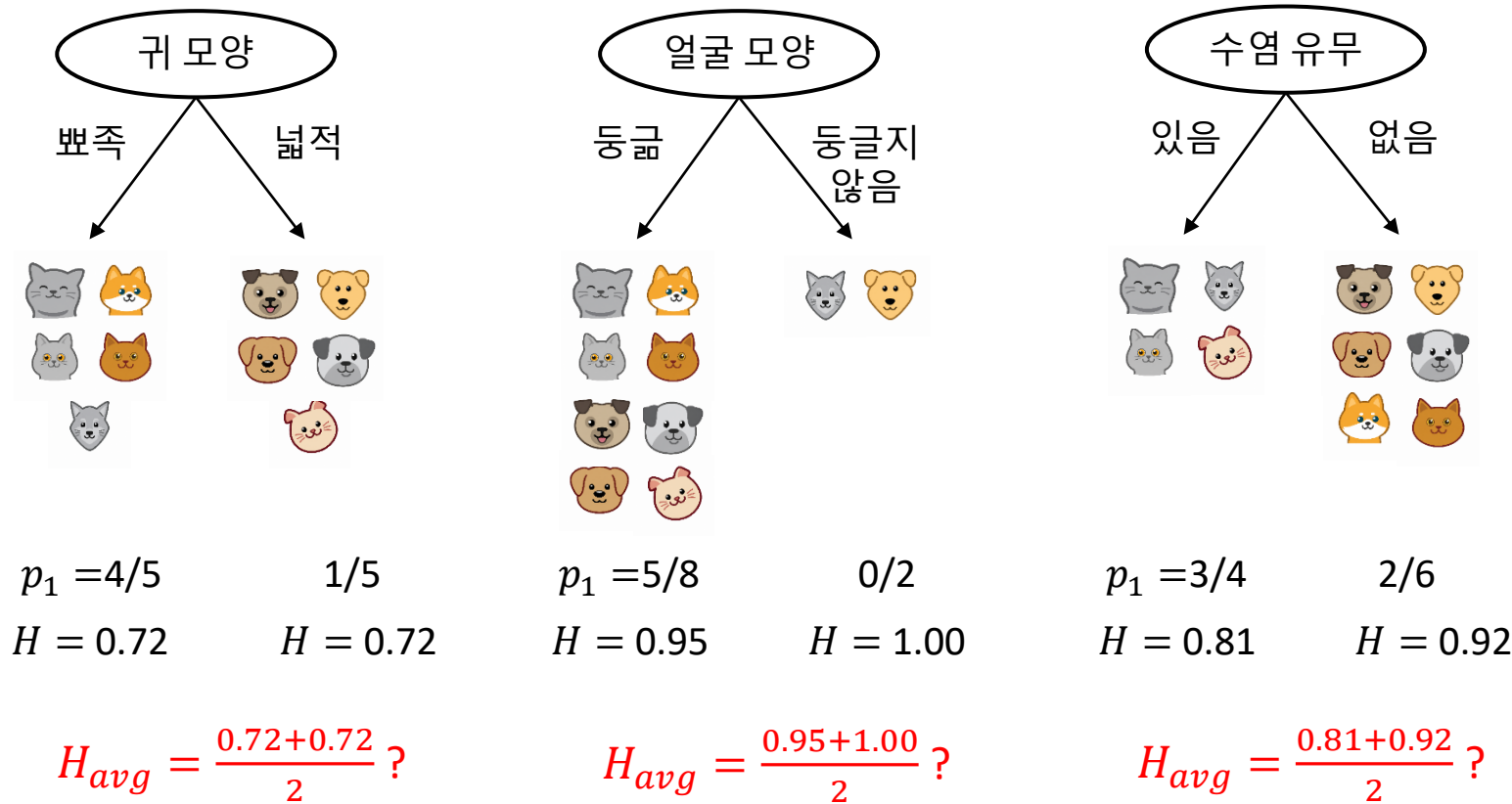
$$G = 1 - p_1^2 - (1 - p_1)^2 = 2p_1(1 - p_1)$$

고양이 샘플 수	고양이 아닌 샘플 수	p_1	$H(p_1)$	$G(p_1)$
2	4	2/6	0.92	0.44
3	3	3/6	1	0.5
5	1	5/6	0.65	0.28
6	0	6/6	0	0
0	6	0	0	0

정보 이득(Information Gain)

노드 분할 기준(1)

특징 별로 분할되는 하위 노드의 엔트로피가 달라짐

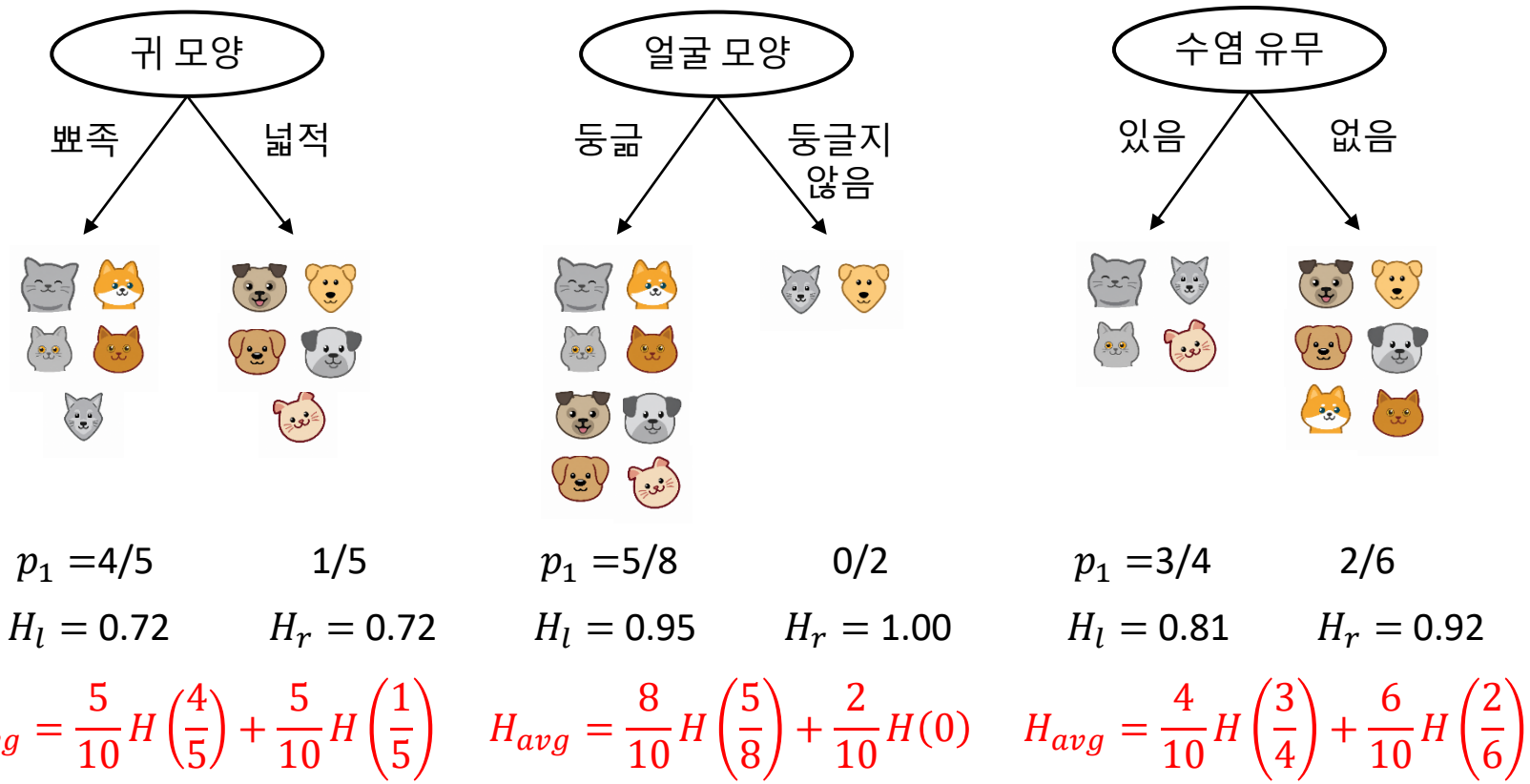


노드 분할 기준(2)

클래스 별 샘플 비율(w^l, w^r)에 따라 가장 평균 엔트로피를 계산

$$H_{avg} = \frac{m^l}{m} H_l + \frac{m^r}{m} H_r$$

m : 현재 샘플 수
 m^l : 분할 후 왼쪽 하위 노드의 샘플 수
 m^r : 분할 후 오른쪽 하위 노드의 샘플 수



정보 이득(Information Gain)

노드 분할 기준(3)

▶ 정보 이득(IG)이 높은 특징을 노드 분할 기준으로 선택

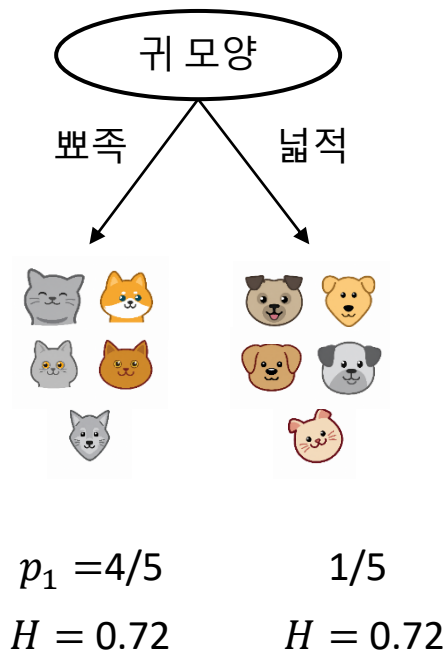
- 엔트로피 변화량이 큼

$$IG = H_{cur} - H_{avg}$$

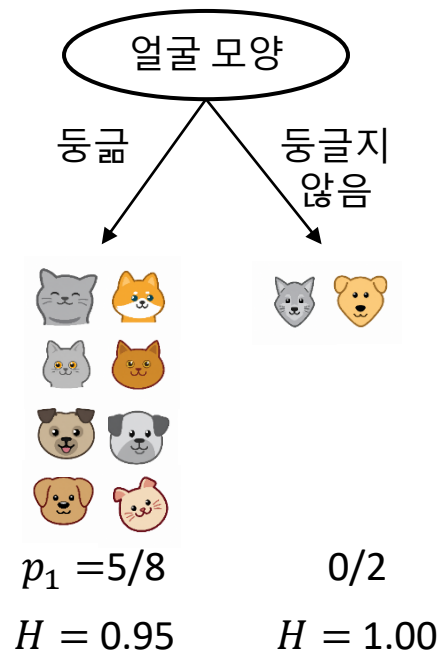
H_{cur} : 분할 전 엔트로피

H_{avg} : 분할 후 하위 노드들의 가중 평균 엔트로피

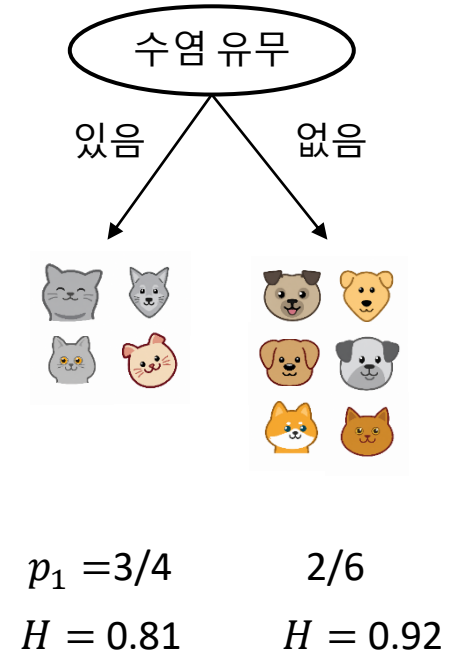
※ H_{cur} 는 1로 가정



$$IG = 1 - 0.72$$



$$IG = 1 - 0.76$$



$$IG = 1 - 0.88$$

■ 전체 훈련 과정(1)

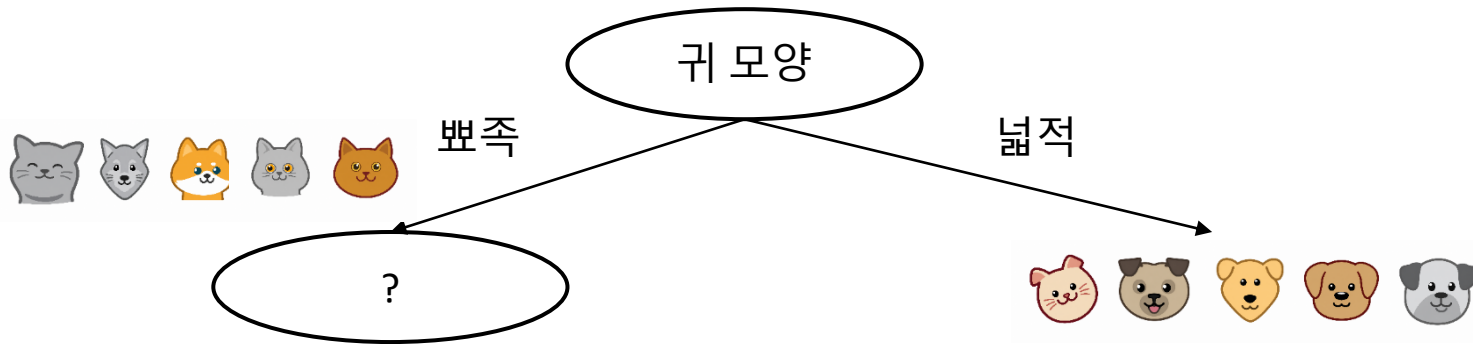
▶ 루트 노드: 특징 별 정보 이득이 가장 높은 특징 선택 (귀 모양, 얼굴 모양, 수염 여부)



■ 전체 훈련 과정(2)

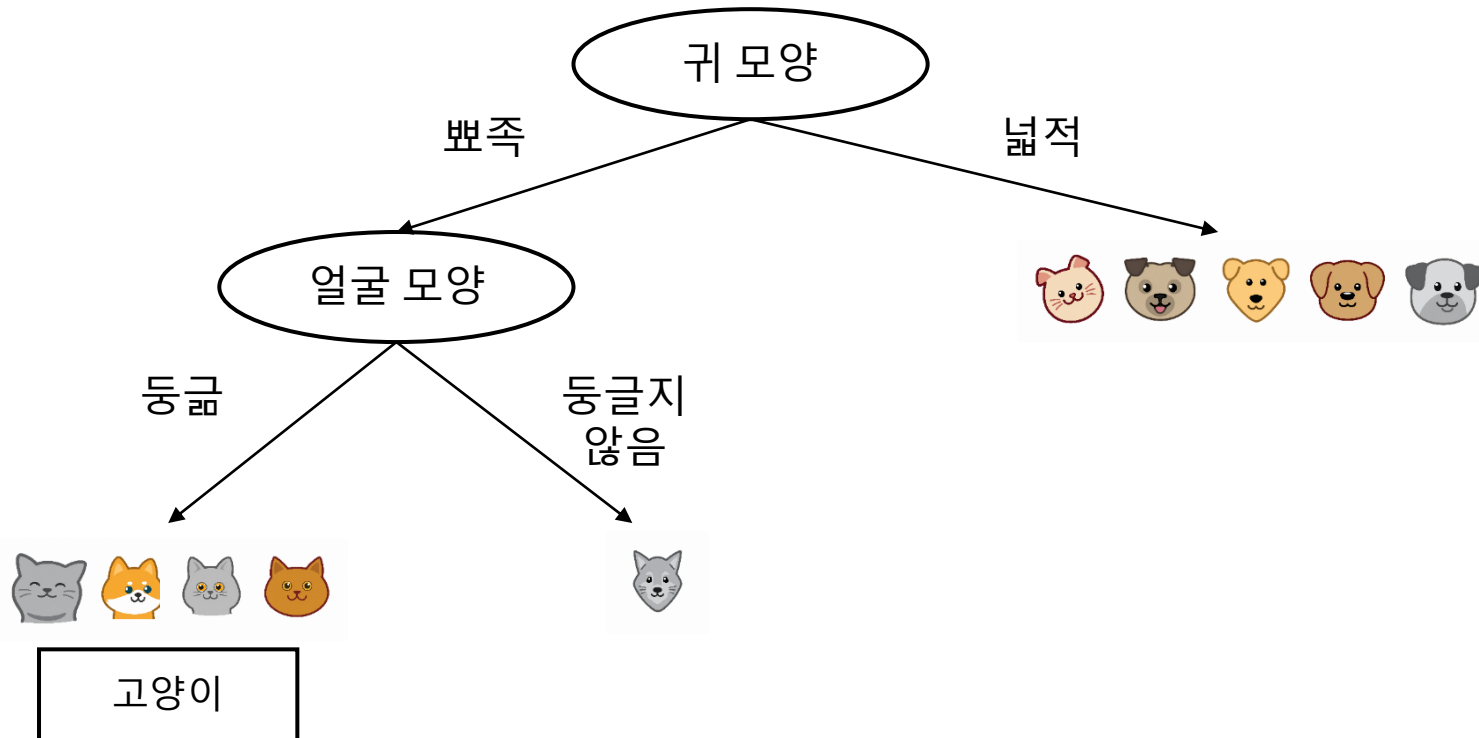
▶ 결정 노드: 분할 특징 별 정보 이득 다시 계산 (얼굴 모양, 수염 여부)

- 가정: 얼굴 모양 IG > 수염 여부 IG



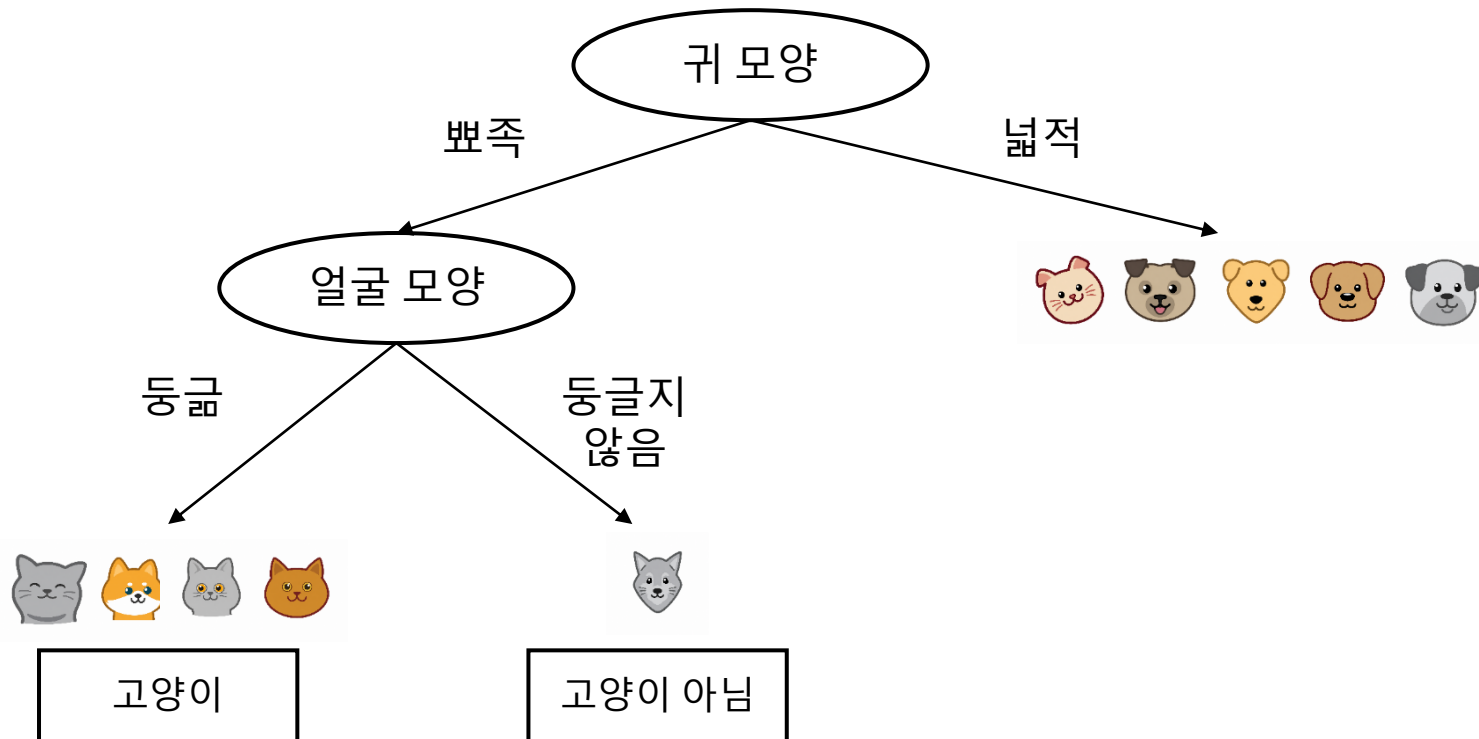
■ 전체 훈련 과정(3)

- ▶ 결정 노드 왼쪽 가지 확인, 순수한 노드가 되면 리프 노드 형성



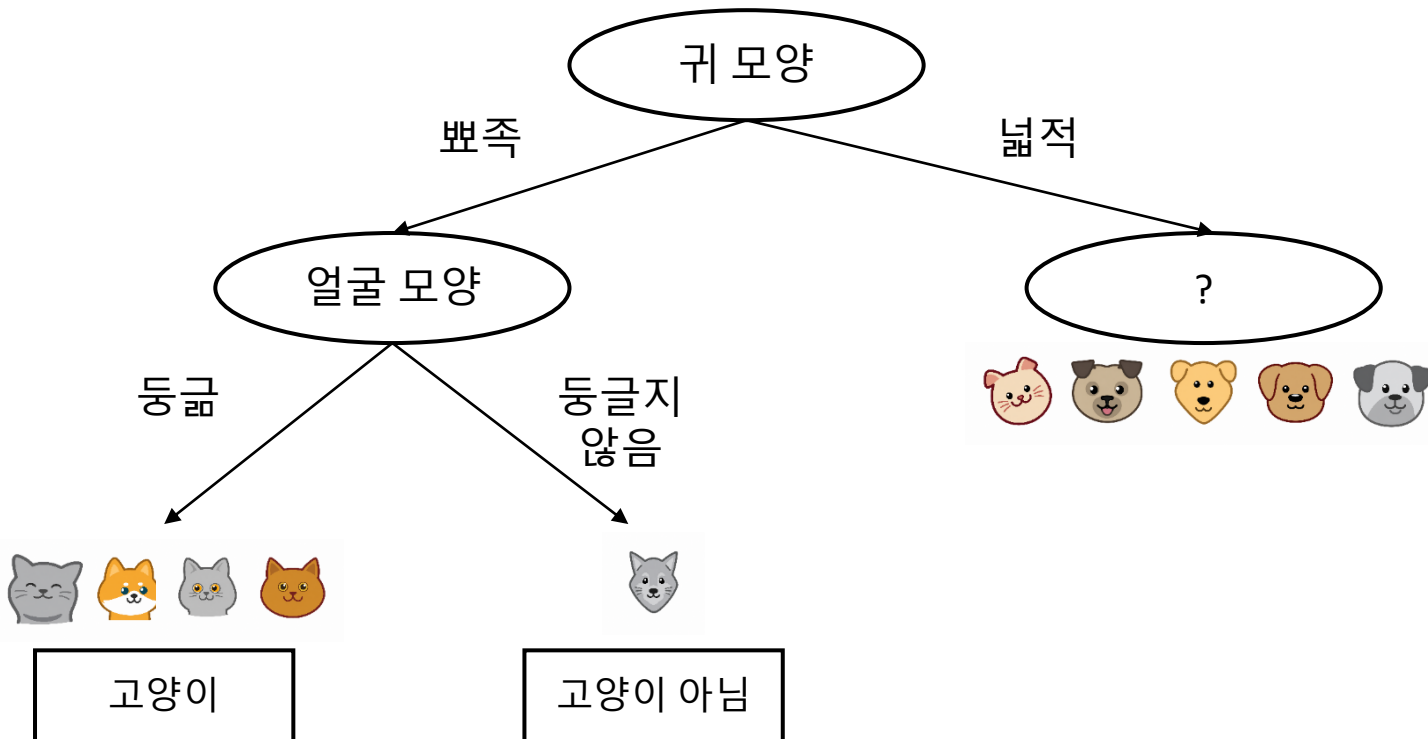
■ 전체 훈련 과정(4)

- ▶ 결정 노드 오른쪽 가지 확인, 순수한 노드가 되면 리프 노드를 만듦



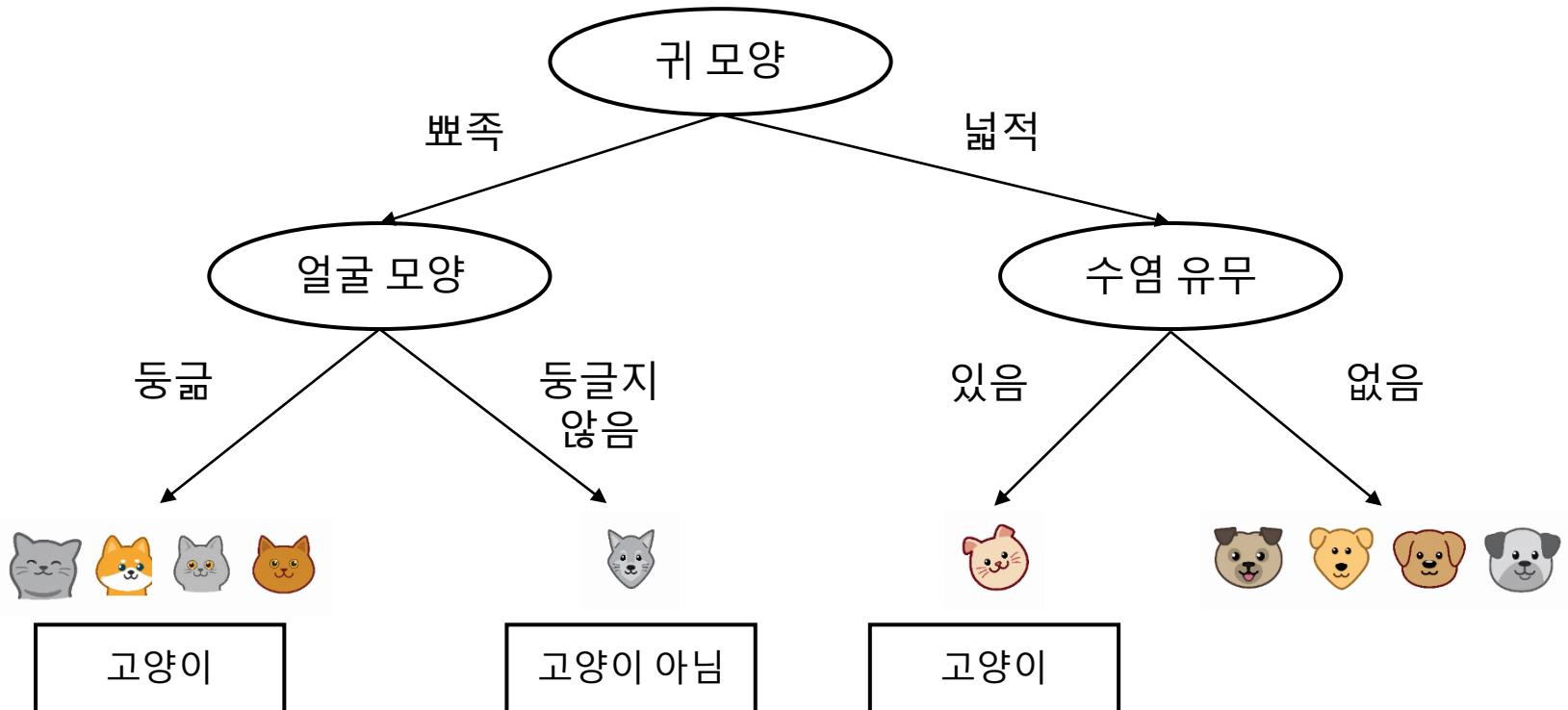
전체 훈련 과정(5)

- ▶ 루트 노드의 오른쪽 가지 확인: 분할 특징 별 정보 이득 다시 계산(얼굴 모양, 수염 유무)
 - 가정: 얼굴 모양 IG < 수염 유무 IG



■ 전체 훈련 과정(6)

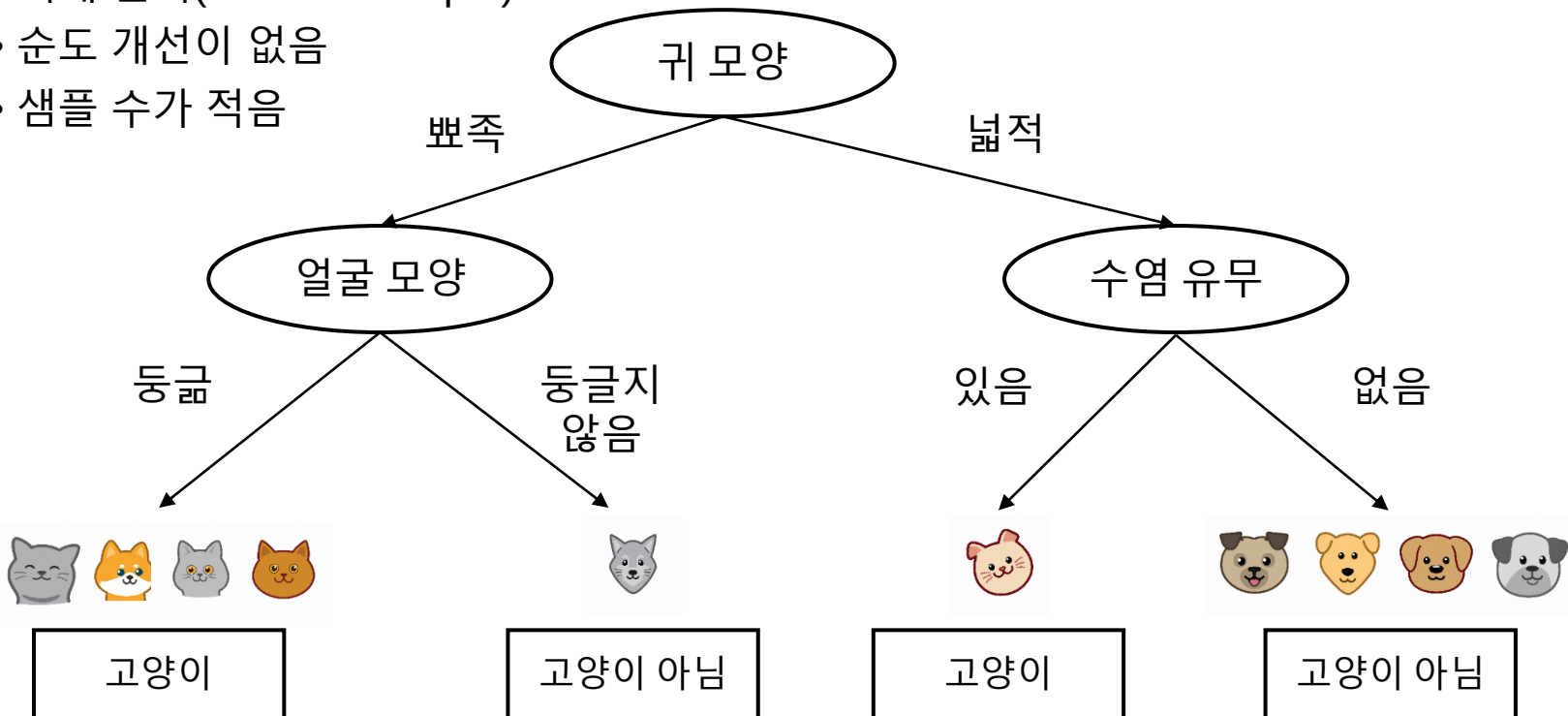
- ▶ 결정 노드 왼쪽 가지 확인, 순수한 노드가 되면 리프 노드 형성



■ 전체 훈련 과정(7)

▶ 결정 노드 기준 오른쪽 가지 확인, 이후 정지 기준을 만족한다면 훈련 종료

- 클래스가 한 개만 남음
- 최대 깊이(maximum depth)
- 순도 개선이 없음
- 샘플 수가 적음



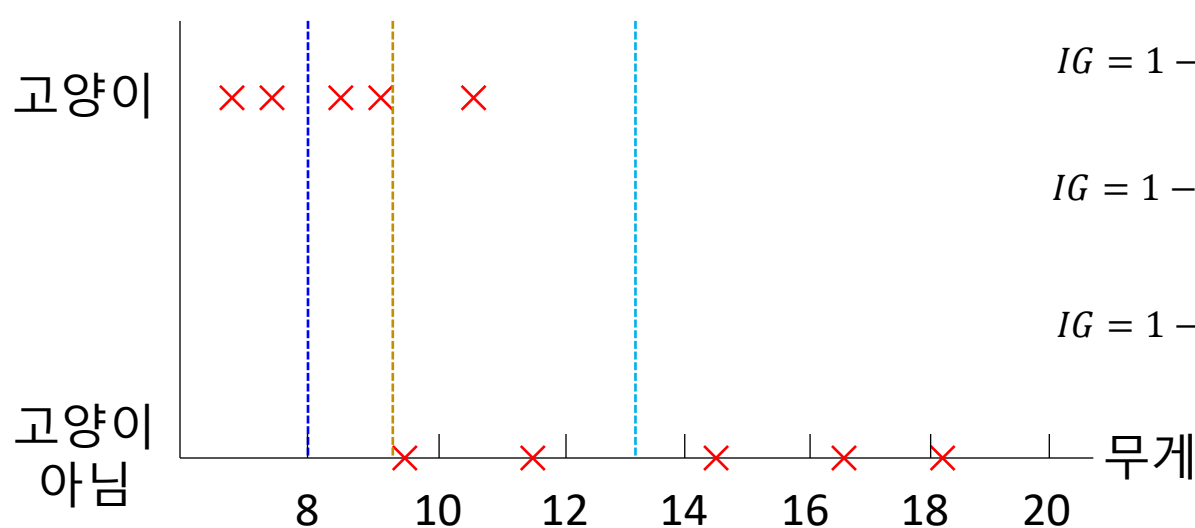
■ 연속형 특징(continuous feature)

▶ 임계값(threshold)을 기준으로 데이터를 분할

- 예시 – 동물 무게 ≤ 8 : 왼쪽 노드, 동물 무게 > 8 : 오른쪽 노드

$$x_j \leq \theta_j$$

▶ 임계값 결정 기준: 여러 임계값들의 정보 이득 계산 $\rightarrow$ 가장 높은 값 선정



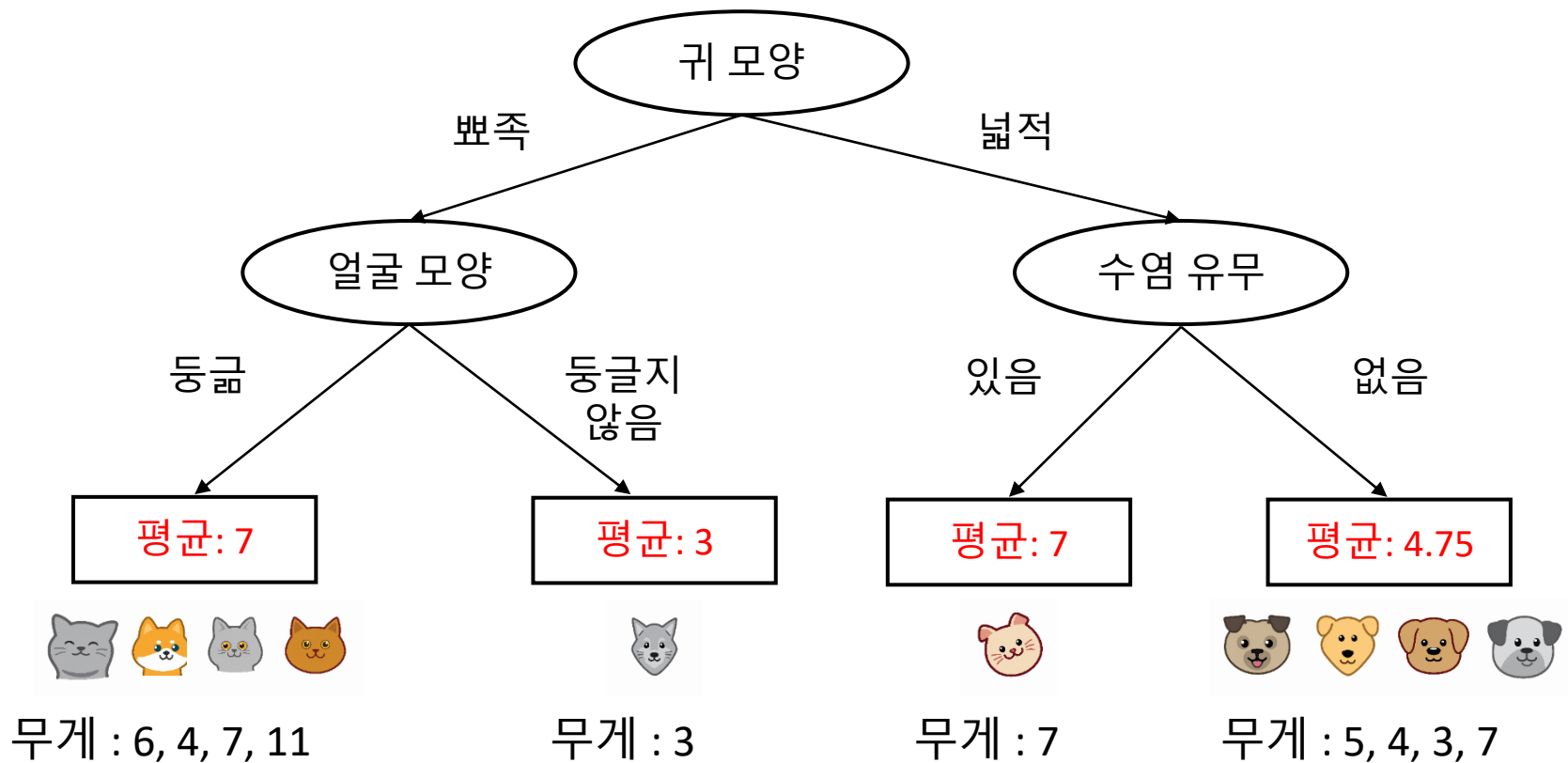
$$IG = 1 - \left(\frac{2}{10} H\left(\frac{2}{2}\right) + \frac{8}{10} H\left(\frac{3}{8}\right) \right) = 0.24$$

$$IG = 1 - \left(\frac{4}{10} H\left(\frac{4}{4}\right) + \frac{6}{10} H\left(\frac{1}{6}\right) \right) = 0.61$$

$$IG = 1 - \left(\frac{7}{10} H\left(\frac{5}{7}\right) + \frac{3}{10} H\left(\frac{0}{3}\right) \right) = 0.40$$

회귀 문제 적용(1)

- ▶ 리프 노드에 도달한 훈련 데이터들의 평균 타깃값을 예측값으로 선정
 - 결국 리프 노드에는 비슷한 타깃값을 가진 샘플들이 모임

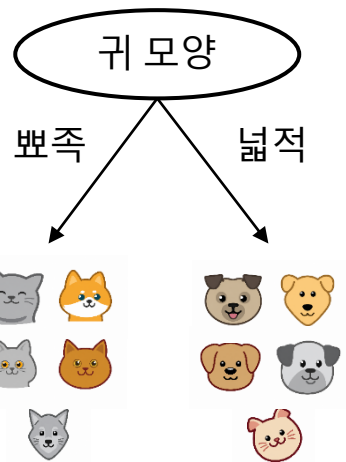


결정 트리(회귀)

회귀 문제 적용(2)

▶ 노드 분할 시 특징 선택 기준은 분산(variance)을 많이 줄이는 특징을 선택

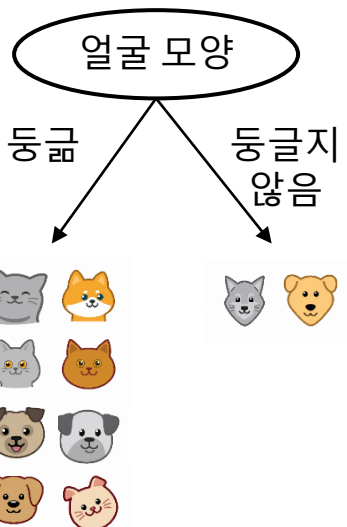
※ 가정: 분할하고자 하는 결정 노드의 분산은 20.51로 가정



무게	6, 4, 7, 11, 3	5, 4, 3, 7, 7
분산	8.16	2.56

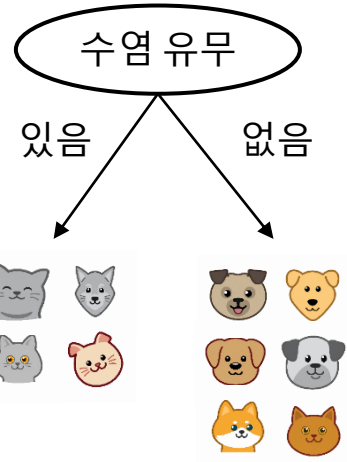
분산 변화

$$20.51 - \left(\frac{5}{10} * 8.16 + \frac{5}{10} * 2.56 \right) = 15.15$$



무게	6, 4, 7, 11, 5, 7, 3, 7	3, 4
분산	5.75	0.25

$$20.51 - \left(\frac{8}{10} * 5.75 + \frac{2}{10} * 0.25 \right) = 15.86$$

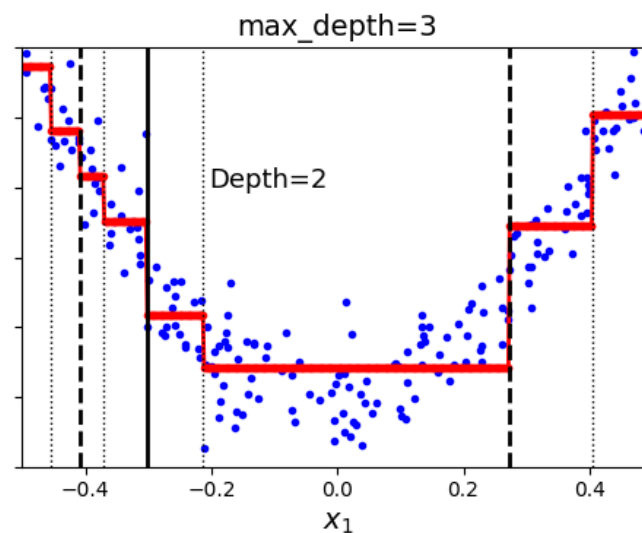
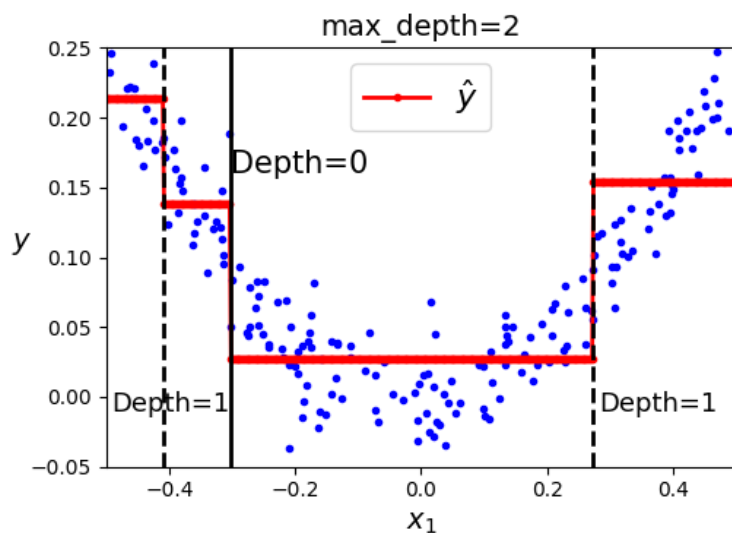


무게	6, 3, 7, 7	5, 4, 3, 7, 4, 11
분산	2.6875	6.89

$$20.51 - \left(\frac{4}{10} * 2.6875 + \frac{6}{10} * 6.89 \right) = 15.30$$

회귀 문제 적용(3)

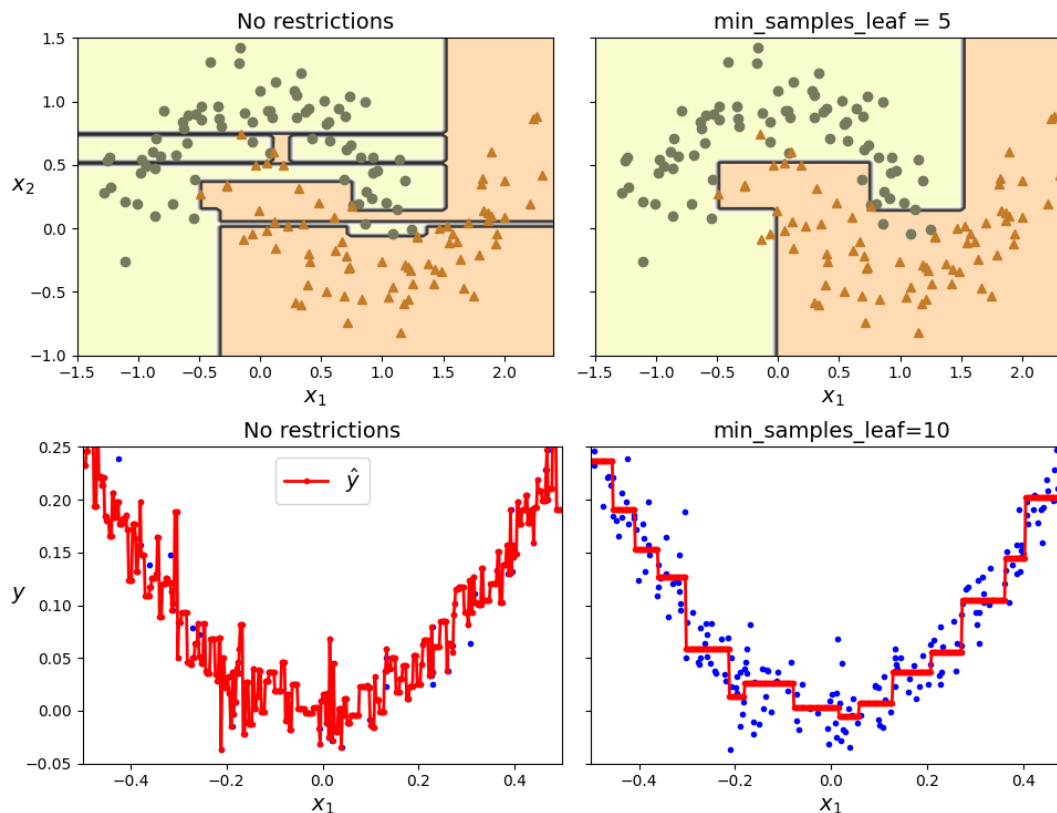
- 특징 공간을 여러 구간으로 나누고, 각 구간 훈련 데이터의 평균값을 예측값으로 산출



하이퍼파라미터 종류

▶ 결정 트리의 복잡도를 규제(regularization)하여 과적합, 과소적합 방지

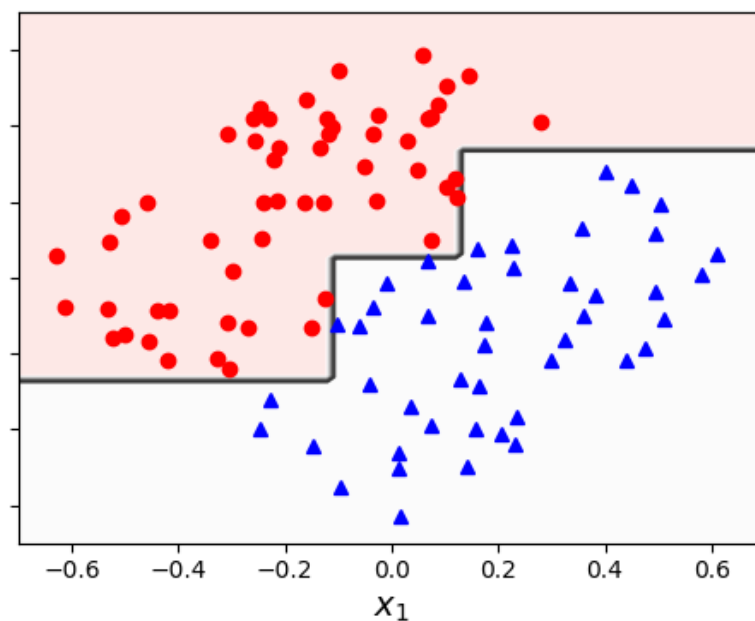
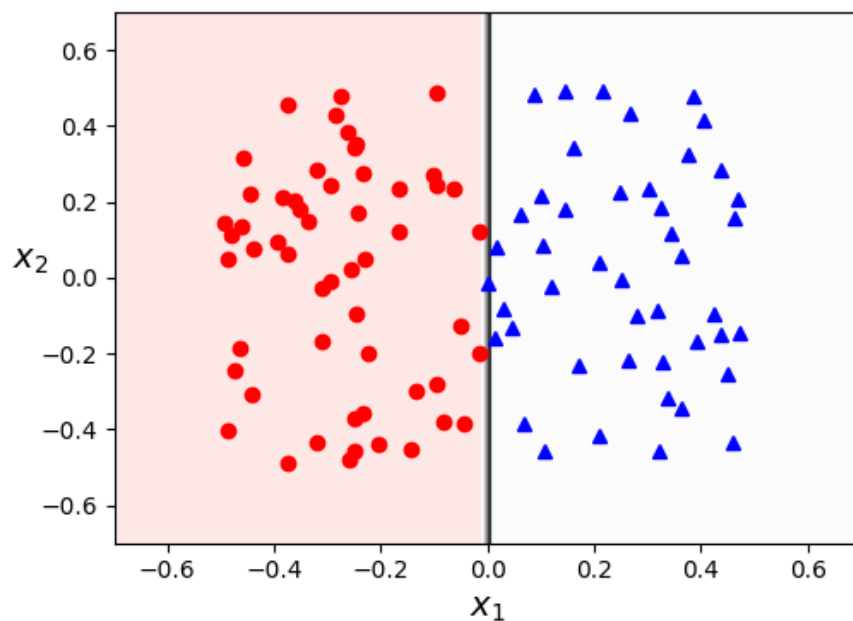
- `max_features` : 각 분할에서 고려할 최대 특징 수
- `max_leaf_nodes` : 트리 전체에서 만들 수 있는 리프 노드의 최대 수
- `min_samples_split` : 분할되기 위해 노드가 가져야 하는 최소 샘플 수
- `min_samples_leaf` : 리프 노드가 생성되기 위해 가지고 있어야 할 최소 샘플 수



결정 트리의 한계(1)

▶ 데이터의 방향성에 대해 민감함

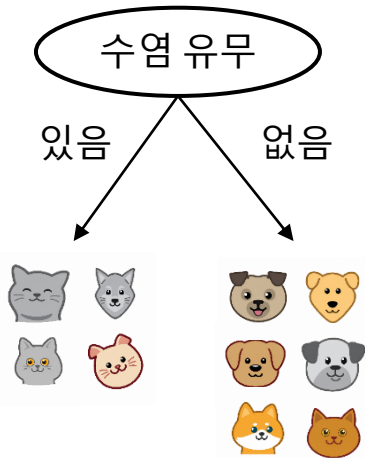
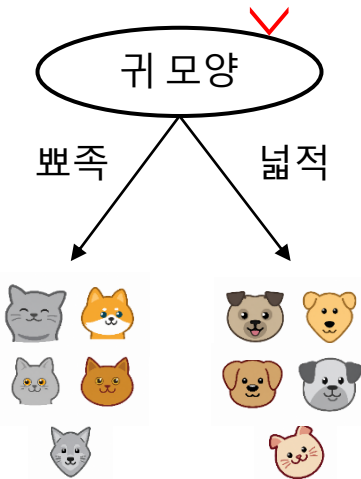
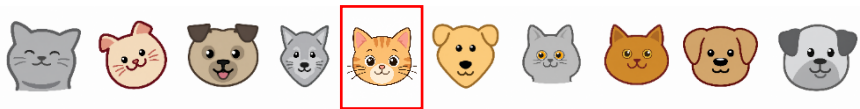
- 계단 모양의 결정 경계를 만들
- 주성분 분석(PCA) 같은 기법을 통해 특징 공간을 변형 후 활용하면 보완 가능



결정 트리의 한계(2)

▶ 단일 결정 트리는 데이터 변화에 민감함

- 훈련 데이터가 조금만 변해도 결정 트리의 구조가 크게 바뀔 수 있음
→ 안정성이 낮다 (robust 하지 않다)

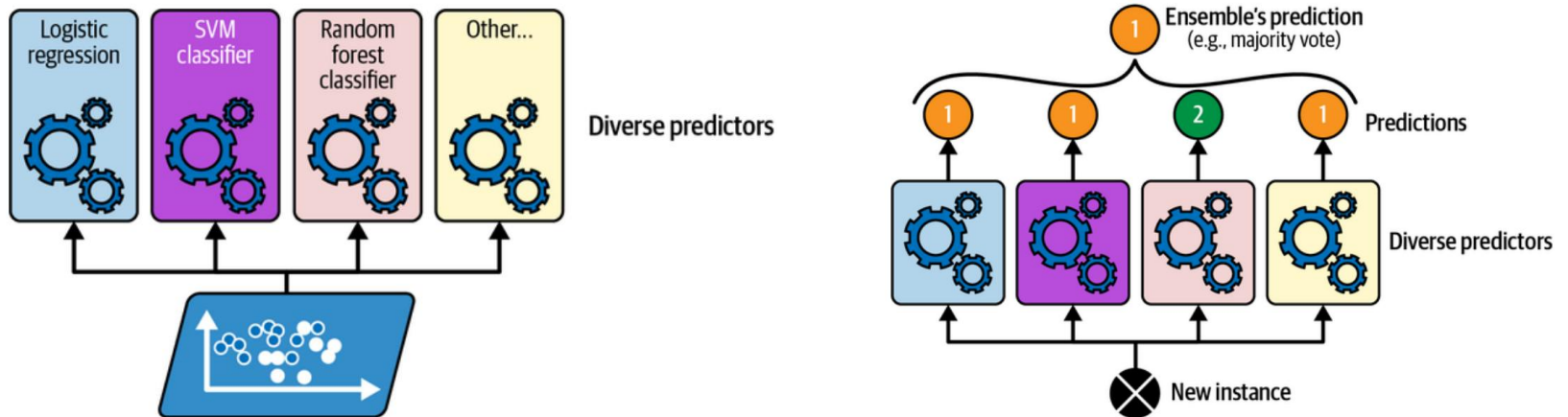


앙상블(Ensemble)

앙상블(ensemble)

- ▶ 여러 개의 예측기(분류기)를 훈련하고, 예측을 결합하여 최종 예측을 도출하는 방법
 - 앙상블 방법 : 보팅(voting), 배깅(bagging), 부스팅(boosting), 스택킹(stack)

<Voting: 다수결의 원칙 (hard voting)>



각 모델 예측값(확률)의 평균을 최종 예측값(확률)으로 선택(soft voting)

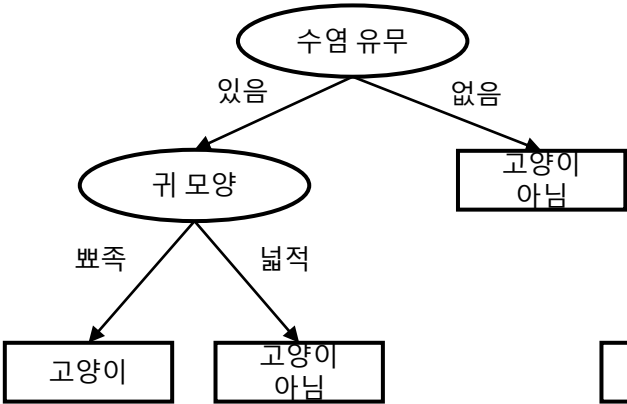
앙상블(Ensemble)

예시. 하드 보팅(Hard voting)

여러 개의 결정 트리를 훈련 후 예측에 사용

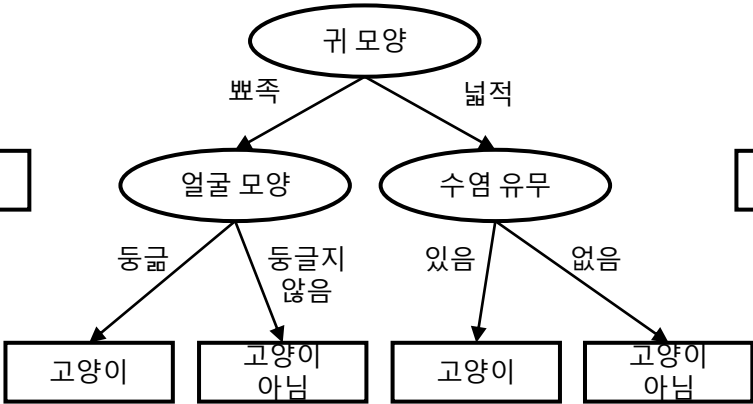


결정 트리 1



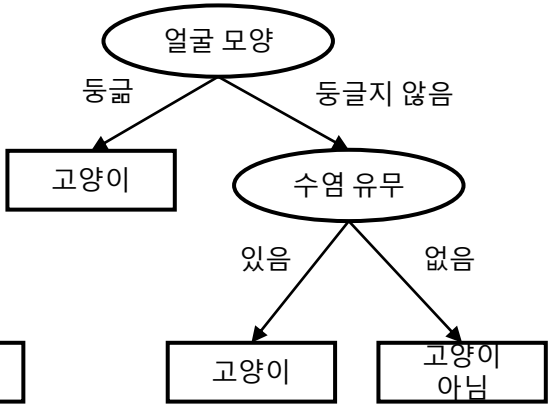
예측: 고양이

결정 트리 2



예측: 고양이 아님

결정 트리 3



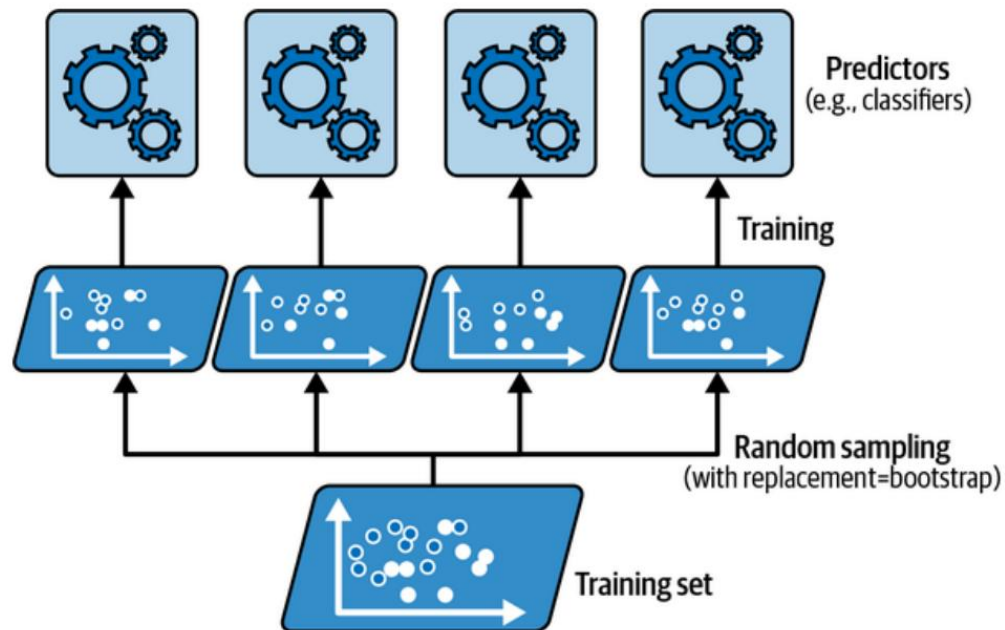
예측: 고양이

최종 예측: 고양이

■ 배깅(bagging, bootstrap aggregating)

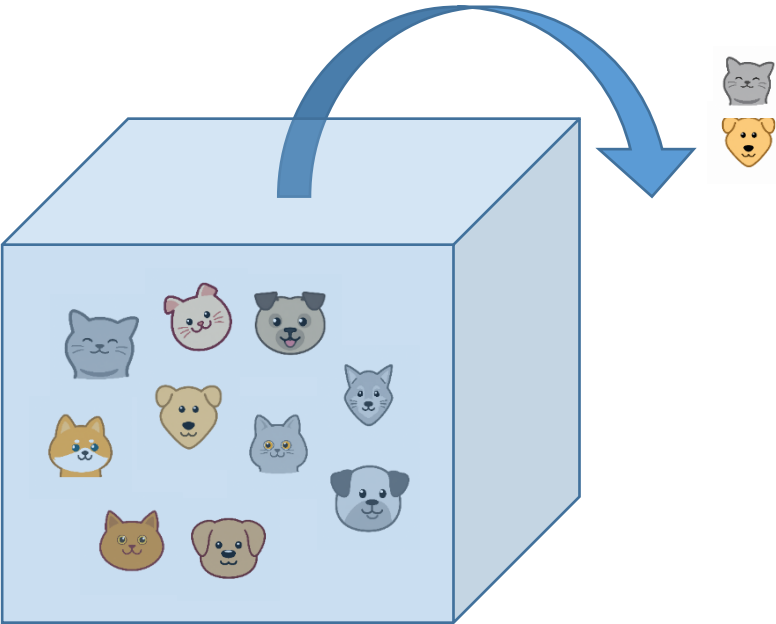
▶ 훈련 세트에서 중복을 허용하여 샘플링 후 각 예측기를 학습

- 같은 훈련 샘플을 여러 개의 예측기 학습에 사용할 수 있음
- 위의 샘플링 방법을 “부트스트래핑(bootstrapping)”이라고 함



예시. 부트스트래핑(Bootstrapping)

- ▶ 원본 훈련 세트에서 중복을 허용하여 여러 개의 훈련 세트를 리샘플링(resampling)
 - 활용 예시: 다양한 결정 트리 훈련 가능



부트스트랩 1

귀 모양	얼굴 모양	수염 유무	고양이
뽀족	둥글	있음	1
넓적	둥글지 않음	없음	0
뽀족	둥글	있음	1
넓적	둥글	있음	1
⋮	⋮	⋮	⋮
뽀족	둥글지 않음	있음	1

⋮

부트스트랩 N

귀 모양	얼굴 모양	수염 유무	고양이
넓적	둥글지 않음	없음	0
넓적	둥글지 않음	없음	0
넓적	둥글	있음	0
뽀족	둥글	있음	1
⋮	⋮	⋮	⋮
뽀족	둥글지 않음	있음	1

랜덤 포레스트(Random forest) (1)

▶ 배깅 결정 트리(bagged decision tree)를 생성

- 한계 : 여러 트리가 비슷한 구조를 가질 수 있음 (예: 어떤 특징이 강력한 경우)

<Bagged decision tree>

훈련 세트 샘플 수 (m)

For $b = 1$ to B (트리 개수) :

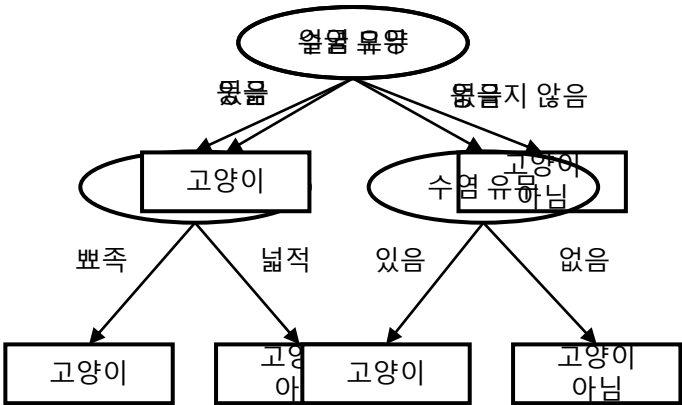
부트스트래핑을 활용해 샘플이 m 개인 훈련 세트 생성

새로운 훈련 세트를 활용해 결정 트리 훈련

부트스트랩 샘플 1

귀 모양	얼굴 모양	수염 유무	고양이
넓적	둥글지 않음	없음	0
넓적	둥글지 않음	없음	0
넓적	둥글	있음	0
뾰족	둥글	있음	1
⋮	⋮	⋮	⋮
뾰족	둥글지 않음	있음	1

결정 트리 1



■ 랜덤 포레스트(Random forest) (2)

- ▶ 각 노드에서 전체 특징을 모두 보지 않고, 무작위로 선택한 일부 특징만 후보로 사용
 - 예시: 전체 특징 N 개, 각 노드에선 K 개 특징만 선택 ($K = \sqrt{N}$)

<전체 특징>

귀 모양
얼굴 모양
수염 유무
특징 1
특징 2
특징 3

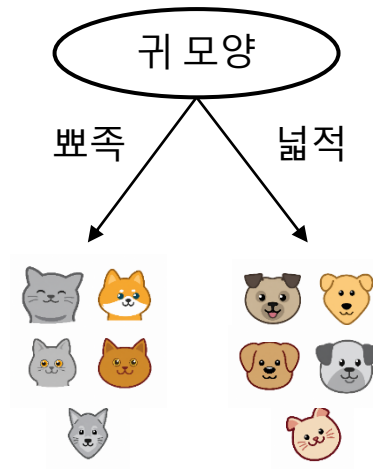


<선택 특징>

귀 모양
특징 2



<배깅 결정 트리 훈련에 반영>



앙상블(Ensemble)

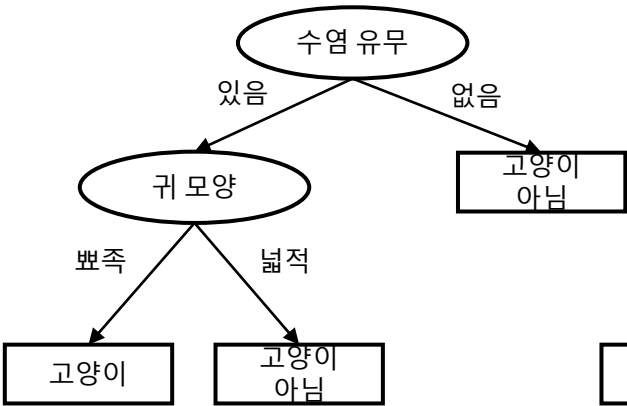
랜덤 포레스트(Random forest) (2)

배깅 결정 트리를 활용해 보팅(voting) 앙상블

<랜덤 포레스트가 안정적(robust)인 이유>

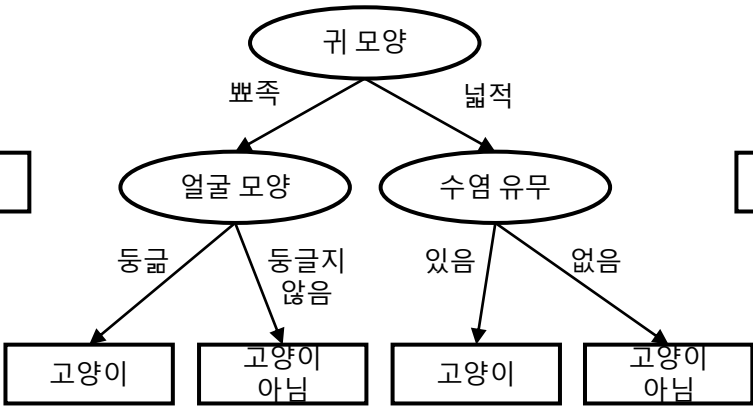
이유 1. 부트스트래핑 방식으로 조금씩 다른 데이터셋 사용
이유 2. 각 노드에서 일부 특징만 사용하므로 트리 구조가 다양해짐

배깅 결정 트리 1



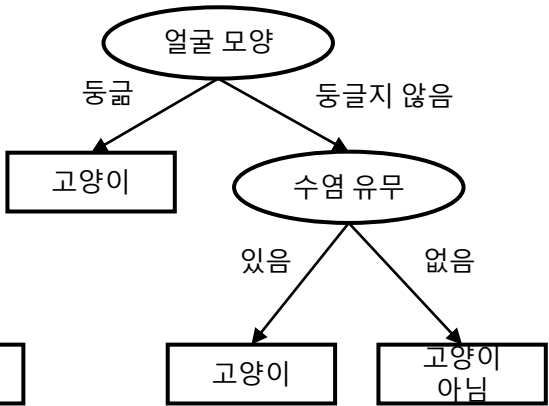
예측: 고양이

배깅 결정 트리 2



예측: 고양이 아님

배깅 결정 트리 3



예측: 고양이

최종 예측: 고양이

앙상블(Ensemble)

부스팅(Boosting)

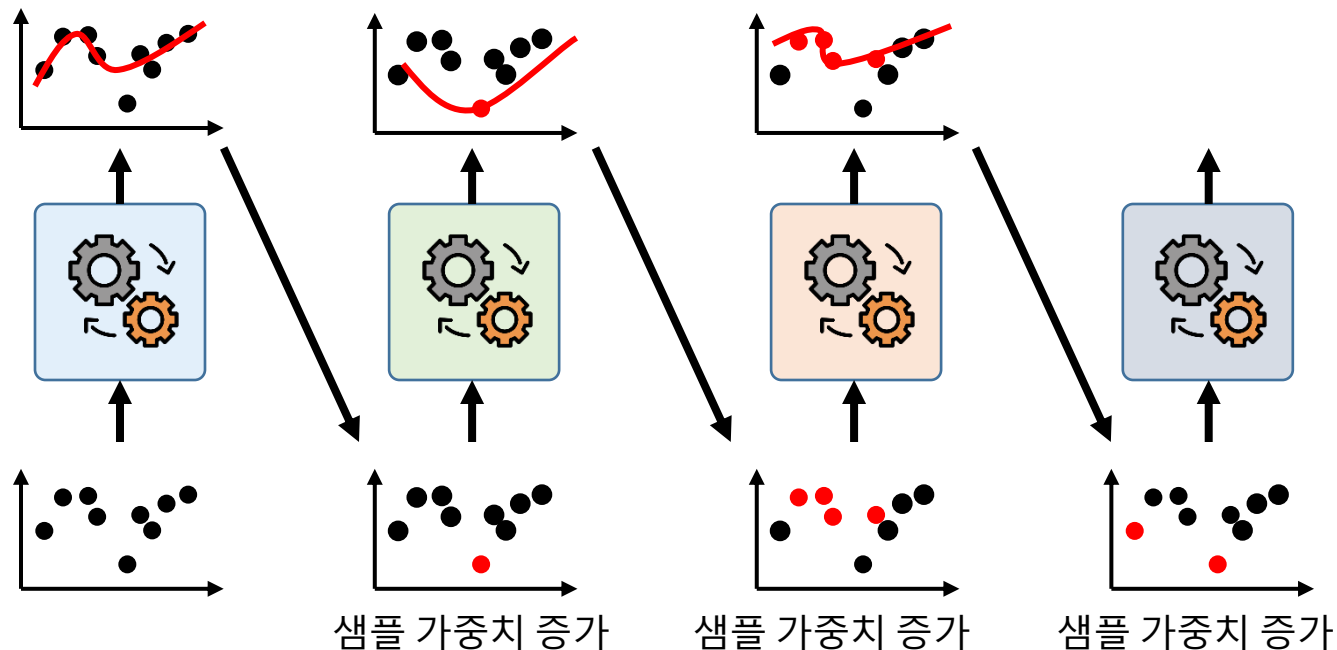
▶ 약한 학습기를 여러 개 연결하여 강한 학습기를 만드는 앙상블 방법

- 약한 학습기(weak learner) : 무작위로 예측하는 것 보다 성능이 조금 좋은 모델
- 강한 학습기(strong learner) : 높은 예측 성능을 가지는 예측 모델

에이다 부스트(AdaBoost, adaptive boosting)

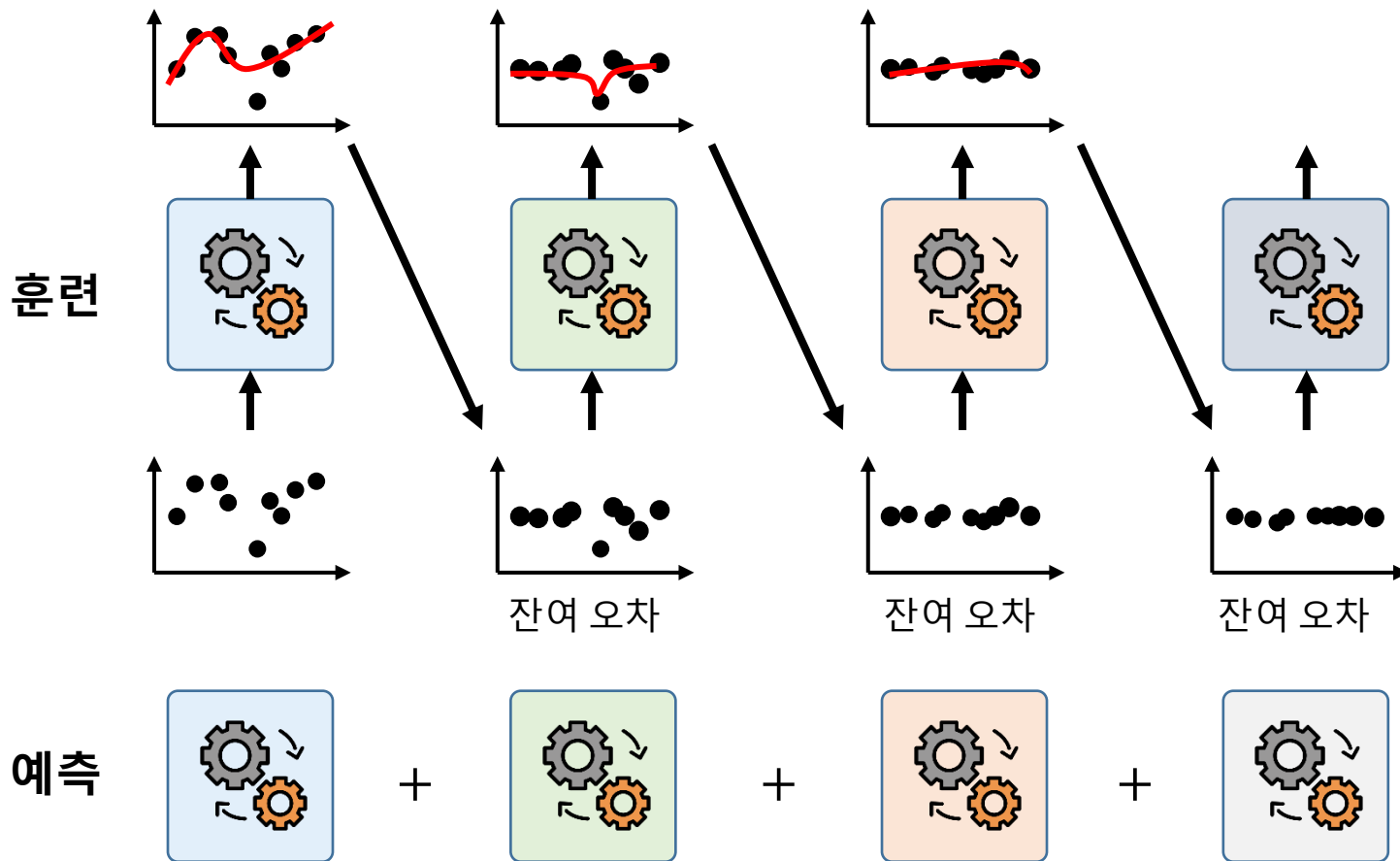
▶ 이전 예측기가 과소적합했던 훈련 샘플의 가중치를 높여서 새로운 예측기가 학습

- 학습하기 어려운 샘플에 점점 더 맞춰짐



▮ 그래디언트 부스팅(Gradient boosting)

▶ 이전 예측기가 만든 잔여 오차(residual error)에 대해 새로운 예측기를 학습

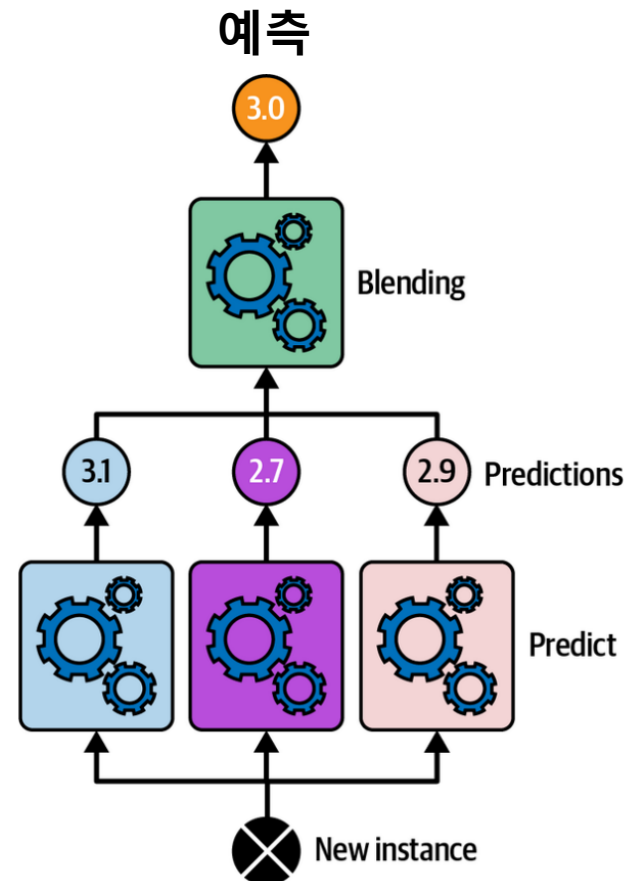
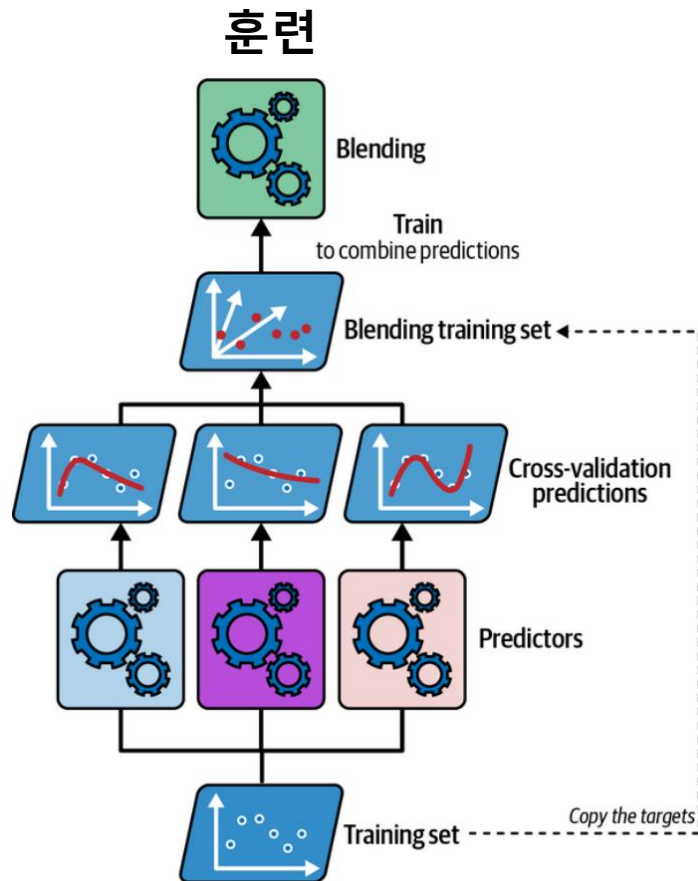


앙상블(Ensemble)

■ 스택킹(Stacking)

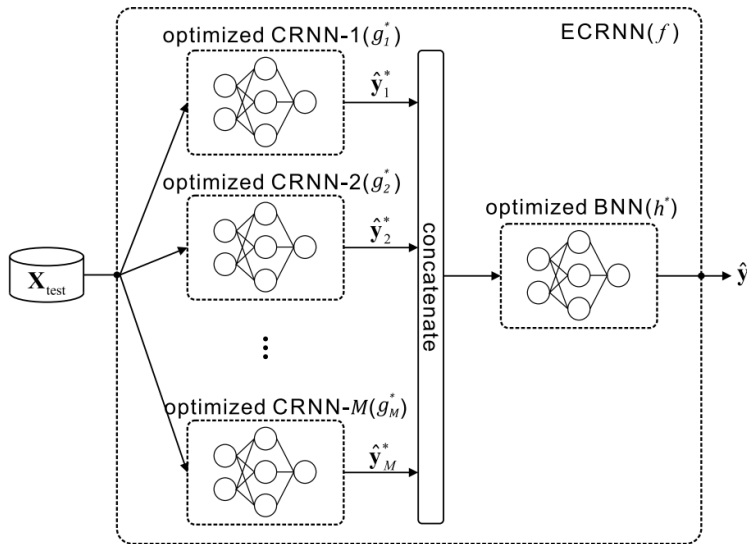
▶ 앙상블에 속한 예측기들의 예측을 취합하는 모델을 훈련시킬 수 있을까?

- 블렌더(blender), 메타러너(meta learner): 다른 모델의 예측을 입력 받아 최종 예측하는 모델

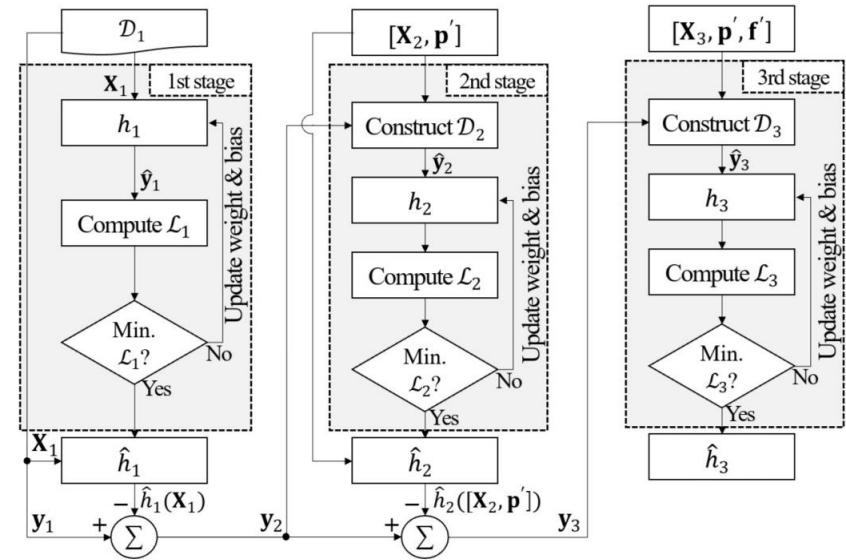


전파·통신분야 응용 사례

Stacking 기반 알고리즘 사례 [1]



Boosting 기반 알고리즘 사례 [2]



- [1] E. Noh et al., "Censored path loss prediction with stacked ensemble of deep neural networks," *IEEE Transactions on Antennas and Propagation*, vol. 74, no. 5, pp. 4700–4712, May 2026. [Top-Tier, JCR Q1 15.6%]
- [2] E. Noh et al., "Sequential residual learning for path loss and shadowing prediction using multimodal data," *IEEE Wireless Communications Letters*, vol. 15, no. 1, pp. 1966–1970, Feb. 2026. [Top-Tier, JCR Q1 15.3%]

I 트리 기반 지도 학습 모델

- ▶ Decision tree
- ▶ Random forest

I 앙상블

- ▶ Voting
- ▶ Bagging
- ▶ Boosting – 예: AdaBoost, Gradient boosting
- ▶ Stacking

실습

■ 실습 준비

- ▶ LMS에 업로드 된 파일 다운로드
- ▶ 프로젝트 폴더 열기
- ▶ 가상환경 설정 확인

I 결정 트리 기반 분류

▶ DecisionTreeClassifier를 활용한 분류 문제 적용

- `from sklearn.tree import DecisionTreeClassifier`
- `DecisionTreeClassifier(max_depth=__, random_state=__)`
- `fit..^^`
- `predict..^^`

실습 1-2. 결정 트리(분류)

결정 트리 하이퍼파라미터(분류)

▶ DecisionTreeClassifier를 활용한 하이퍼파라미터 개념 확인

- `max_depth` = [None, 0, 1, 2, 3]
- `min_sample_leaf` = [1, 2, 3, 4, 5]
- 각 분류기의 score 확인 → 객체명.score(X, y) 메서드 활용
- `max_depth`가 커질수록 예측 결과가 어떻게 변하나요?
- `min_sample_leaf`가 커질수록 예측 결과가 어떻게 변하나요?

실습 1-3. 결정 트리(회귀)

I 결정 트리 기반 회귀

▶ DecisionTreeRegressor를 활용한 분류 문제 적용

- `from sklearn.tree import DecisionTreeRegressor`
- `DecisionTreeRegressor(max_depth=__, random_state=__)`
- `fit..^^`
- `predict..^^`

실습 1-4. 결정 트리(회귀)

I 결정 트리 하이퍼파라미터(회귀)

▶ DecisionTreeRegressor를 활용한 하이퍼파라미터 개념 확인

- max_depth = [None, 0, 1, 2, 3]
- min_sample_leaf = [2, 4, 6, 8, 10]
- 각 예측기의 RMSE 확인
- max_depth가 커질수록 예측 결과가 어떻게 변하나요?
- min_sample_leaf가 커질수록 예측 결과가 어떻게 변하나요?

실습 2-1. 랜덤 포레스트(분류)

■ 랜덤 포레스트(분류)

▶ RandomForestClassifier를 활용한 분류 문제 적용

- 실습 1-1과 동일한 데이터셋 사용
- `from sklearn.ensemble import RandomForestClassifier`
- `fit..^^`
- `predict..^^`

▶ 결정 트리(실습 1-1)와 랜덤 포레스트의 분류 성능을 비교해보세요

- 혼동 행렬 확인(`from sklearn.metrics import confusion_matrix`)
- F1 스코어 확인(`from sklearn.metrics import f1_score`)
- 분류 리포트 확인(`from sklearn.metrics import classification_report`)
- 클래스 사용법 : 클래스명(테스트 데이터, 예측 데이터)

실습 2-2. 랜덤 포레스트(분류)

■ 랜덤 포레스트 하이퍼파라미터

▶ RandomForestClassifier를 활용한 하이퍼파라미터 개념 확인

- `n_estimator` : 배깅 결정 트리 분류기 갯수
- `max_leaf_nodes` : 배깅 결정 트리의 최대 리프 노드 개수
- `n_estimator = [100, 200, 300, 400]`가 커질수록 예측 결과가 어떻게 변하나요?
- `max_leaf_nodes = [2, 4, 8, 16]`가 커질수록 예측 결과가 어떻게 변하나요?

실습 2-2. 랜덤 포레스트(회귀)

■ 랜덤 포레스트(회귀)

▶ RandomForestRegressor를 활용한 분류 문제 적용

- 실습 1-3과 동일한 데이터셋 사용
- 하이퍼파라미터는 사용자가 원하는데로 설정해보세요
- `from sklearn.ensemble import RandomForestRegressor`
- `fit..^^`
- `predict..^^`

▶ 결정 트리(실습 1-3)와 랜덤 포레스트의 회귀 성능을 비교해보세요

- RMSE 확인

■ 랜덤 포레스트 모델 튜닝(분류)

▶ RandomForestClassifier 모델 튜닝

- 붓꽃(Iris) 데이터셋 활용 – Virginica 이진 분류
- `from sklearn.model_selection import GridSearchCV`
- 모델 정의(RandomForestClassifier)
- 탐색하고자 하는 하이퍼파라미터 그리드 생성(`n_estimators`, `max_depth`)
- 그리드 서치 객체 정의
`GridSearchCV(모델,
 하이퍼파라미터 그리드,
 cv=__,
 scoring=“__”)`
- 참고) `scoring`: “score”, “accuracy”, “precision”, “recall”, “f1” 등
- 그리드 서치 객체 `.fit(X, y)`

■ 랜덤 포레스트 모델 튜닝(회귀)

▶ RandomForestRegressor 모델 튜닝

- 캘리포니아 주택 가격 데이터셋 사용
- `from sklearn.model_selection import GridSearchCV`
- 모델 정의(RandomForestRegressor)
- 탐색하고자 하는 하이퍼파라미터 그리드 생성(`n_estimators`, `max_depth`)
- 그리드 서치 객체 정의
`GridSearchCV(모델,
 하이퍼파라미터 그리드,
 cv=__,
 scoring="neg_root_mean_squared_error")`
- 그리드 서치 객체 `.fit(X, y)`

하드 보팅

▶ 로지스틱 회귀, 랜덤 포레스트, 서포트 벡터 머신으로 세부 예측기 구성

- `from sklearn.ensemble import VotingClassifier`
- `make_moons` 데이터셋 생성(`n_samples=500`, `noise=0.30`, `random_state=42`)
- 훈련, 테스트 데이터 분할(`random_state=42`)
- `VotingClassifier` 객체 생성
- `VotingClassifier(`
 `estimators=[('예측기 이름', 모델),`
 `( '예측기 이름', 모델),`
 `( '예측기 이름', 모델)]`
 `)`
- 보팅 분류기 객체.`fit(X, y)`
- 보팅 분류기 객체.`predict(X, y)`
- 보팅 분류기 객체.`score(X, y)` → 보팅 분류기 성능 확인

배깅

▶ BaggingClassifier를 활용한 결정 트리 앙상블 훈련

- `from sklearn.ensemble import BaggingClassifier`
- `BaggingClassifier(모델, n_estimators=__, max_samples=__)`
- 참고) `max_samples`: 전체 훈련 세트 샘플 대비 부트스트랩 세트의 샘플 비율(0~1)
- 배깅 객체.`fit(X, y)`
- 배깅 객체.`predict(X, y)`

부스팅(1)

▶ AdaBoostClassifier를 활용한 결정 트리 앙상블 훈련

- `from sklearn.ensemble import AdaBoostClassifier`
- `AdaBoostClassifier(모델, n_estimators=__, learning_rate=__)`
- 참고) `learning_rate`: 최종 예측 시 각 학습기의 영향력을 얼마나 크게 반영할지 결정
- 부스팅 객체.`fit(X, y)`
- 부스팅 객체.`predict(X, y)`

I 부스팅(2)

▶ GradientBoostingRegressor를 활용한 결정 트리 앙상블 훈련

- `from sklearn.ensemble import GradientBoostingRegressor`
- `GradientBoostingRegressor(max_depth=__, n_estimators=__, learning_rate=__)`
- 참고) `max_depth`: 각 결정 트리의 깊이
- 참고) `learning_rate`: 최종 예측 시 각 학습기의 영향력을 얼마나 크게 반영할지 결정
- 부스팅 객체.`fit(X, y)`
- 부스팅 객체.`predict(X, y)`

스태킹

▶ 로지스틱 회귀, 랜덤 포레스트, 서포트 벡터 머신 기반 스태킹 앙상블 학습

- `from sklearn.ensemble import StackingClassifier`
- `StackingClassifier(
 estimators=[('예측기 이름', 모델),
 ('예측기 이름', 모델),
 ('예측기 이름', 모델)],
 final_estimator=모델),
 cv=__)`
- 스태킹 분류기 객체.`fit(X, y)`
- 스태킹 분류기 객체.`predict(X, y)`